FIAP ON

IA COMO FERTILIZANTE DIGITAL - UM NOVO AGRONEGÓCIO DO AMANHÃ

AWS EM TEMPO CERTO: PROCESSOS ASSÍNCRONOS, ALERTAS E MONITORAMENTO INTELIGENTE

5

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - Projeto de uma solução de processamento de imagens.....7
Figura 2 - Fila de Mensagens.....9
Figura 3 - Tipos de filas do SQS 10
Figura 4 - Dead Letter Queue.....11
Figura 5 - Mecanismo de *push* 12
Figura 6 - Protocolos de Transporte SNS 13
Figura 7 - Definição da API e função Lambda para integração 15
Figura 8 - Rota para pedidos.....18
Figura 9 - Integração com função Lambda.....18
Figura 10 - *Endpoint* da API 19
Figura 11 - Exemplo de chamada de API.....19
Figura 12 - Criação de tópico e assinatura.....20
Figura 13 - Menu Tópicos – SNS 21
Figura 14 - Criação de tópico pedido-recebido-topico3.....21
Figura 15 - Tópico criado.....21
Figura 16 - Criação de variável de ambiente no Lambda.....22
Figura 17 - Variável de ambiente para o SNS.....22
Figura 18 - Visão Geral de Assinaturas no SNS 23
Figura 19 - Criação de assinatura para e-mail 23
Figura 20 - Assinatura criada 23
Figura 21 - Envio de mensagens via Postman.....24
Figura 22 - E-mail recebido 24
Figura 23 - Criação de pilha SQS.....25
Figura 24 - Criação de Pilha SQS 26
Figura 25 - Criação de fila padrão.....26
Figura 26 - Adição de conta do SNS como produtor 27
Figura 27 - Gatilho para fila SQS 28
Figura 28 - Gatilho para Fila SQS 29
Figura 29 - Envio de nova mensagem à API.....29
Figura 30 - Dashboard do CloudWatch 30
Figura 31 - Lista de Grupos de Logs 31
Figura 32 - Log gerado no CloudWatch 31
Figura 33 - Evento de Log.....32
Figura 34 - Detalhes para cobrança da função Lambda 32
Figura 35 - Evento de processamento concluído 32
Figura 36 - Métricas do CloudWatch.....33
Figura 37 - Dashboard de Métricas.....33
Figura 38 - Dashboard de métricas de funções Lambda.....34
Figura 39 - Criação de alarmes de invocações 34
Figura 40 - Condições do alarme 35
Figura 41 - Criação de tópico para alarme 36
Figura 42 - Definições do alarme 36
Figura 43 - Alarme disparado.....37
Figura 44 - E-mail de notificação de alarme.....37
Figura 45 - Criação de alarme de Erro 38
Figura 46 - Seleção de métrica de alarme 38
Figura 47 - Seleção de métrica da função Lambda.....38

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 48 - Criação de métrica de erros 39

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - recebe-pedidos-api.py 16
Código-fonte 2 - Arquivo JSON de exemplo.....17
Código-fonte 3 - Política de acesso da fila SQS.....27
Código-fonte 4 - processar-pagamento.py 28

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 AWS EM TEMPO CERTO: PROCESSOS ASSÍNCRONOS, ALERTAS E MONITORAMENTO INTELIGENTE....6
1.1 Serviços de enfileiramento e notificação....6
1.2 Projeto Real....6
2 PROCESSAMENTO ASSÍNCRONO E MENSAGERIA....9
2.1 Simple Queue Services (SQS)....9
2.2 Simple Notification Service (SNS)....12
2.3 SNS *versus* SQS....13
2.4 Precificação....14
3 CONSTRUINDO NOSSO PROJETO....15
3.1 Criação de API....15
3.2 Criação do tópico SNS....20
3.3 Criação de pilha SQS....25
3.4 Amazon CloudWatch....30
3.5 Monitoramento com CloudWatch....33
CONCLUSÃO....40
REFERÊNCIAS....41
GLOSSÁRIO....42

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 AWS EM TEMPO CERTO: PROCESSOS ASSÍNCRONOS, ALERTAS E MONITORAMENTO INTELIGENTE

1.1 Serviços de enfileiramento e notificação

A AWS permite processar trabalhos de forma assíncrona usando serviços de mensageria como o Simple Queue Service (SQS), o Simple Notification Service (SNS) e os códigos *serverless* com Lambda.

Utilizando os serviços de mensageria, é possível:

- Criar um fluxo assíncrono de mensagens;
- Garantir a entrega de mensagens pelo menos uma vez;
- Integrar aplicações em um sistema distribuído;
- Comunicar aos assinantes sobre novas mensagens.

Além de criar um fluxo assíncrono, é importante monitorar a execução de serviços no ambiente da AWS. Para isso, veremos como trabalhar com o CloudWatch, que monitora os recursos da AWS além de permitir criar alertas.

1.2 Projeto Real

Temos o seguinte cenário: imagine que você atua em um e-commerce e precisa processar pedidos de forma assíncrona, sinalizando o recebimento dos pedidos e confirmando o pagamento.

A seguir, podemos visualizar o diagrama da solução assíncrona para o processamento de imagens na AWS. Note os serviços envolvidos e as “caixinhas” protegendo esses serviços.

Agora, vamos explicar como configurar cada microsserviço e como eles vão se comunicar.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

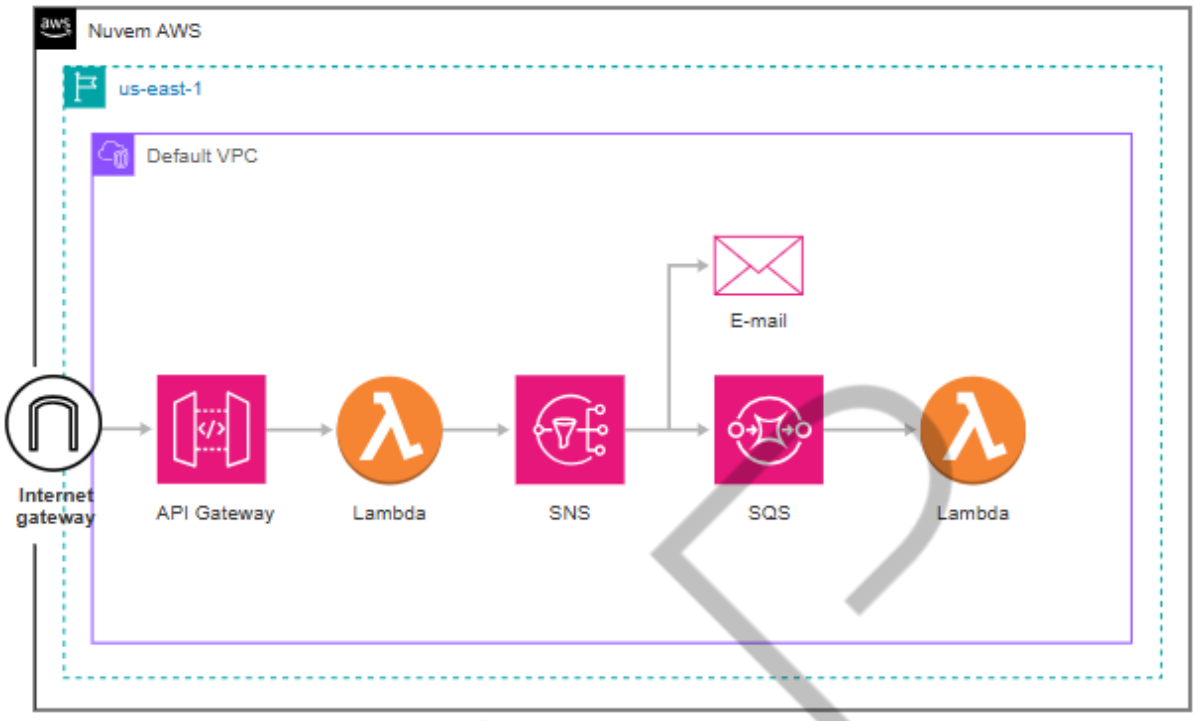

Figura 1 - Projeto de uma solução de processamento de imagens
Fonte: Elaborado pelo autor (2025), adaptado por FIAP (2025)

Para este cenário, precisamos criar uma esteira de processamento assíncrono que irá receber pedidos confirmados de clientes, e que utilizaremos os serviços de mensageria da AWS para comunicação e processamento dos pedidos.

Primeiramente, criaremos uma API para receber os pedidos por meio de um *endpoint* público. Os pedidos serão enviados no formato JSON, contendo os dados do cliente, valores e itens pedidos. A API é um mini software que pode ser altamente customizado, portanto só vai servir para o seu projeto, automatizando uma solução para resolver um problema.

A API receberá os dados do pedido e acionará automaticamente uma função Lambda para processar o pedido. A função Lambda, por sua vez, irá gerar uma mensagem de recebimento de pedido para um **tópico** do **Simple Notification Service (SNS)**, que irá enviar os dados do pedido para seus assinantes.

Agora será preciso criar uma assinatura para confirmar o recebimento do pedido. Em seguida, criar uma fila no **Simple Queue Service (SQS)** para processar o pagamento. Também devemos habilitar o envio de mensagens para o SQS a partir do SNS.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Finalmente, outra função Lambda será acionada para processar o pedido. No nosso caso, para simplificar, o processamento consistirá no envio de uma mensagem de confirmação do pedido, que poderemos monitorar utilizando o serviço **CloudWatch**.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2 PROCESSAMENTO ASSÍNCRONO E MENSAGERIA

Antes de iniciarmos com nosso projeto, iremos abordar os conceitos básicos de mensageria na AWS e suas principais características.

2.1 Simple Queue Services (SQS)

O Amazon Simple Queue Service (SQS) foi um dos primeiros serviços lançados pela AWS. Surgiu em 2006 como parte dos primeiros serviços de computação em nuvem da empresa.

Ele foi projetado para resolver problemas de comunicação assíncrona entre sistemas distribuídos, permitindo o desacoplamento entre componentes de software.

O SQS trabalha com o conceito de **filas**, em que a primeira mensagem que chega é a primeira a ser consumida.

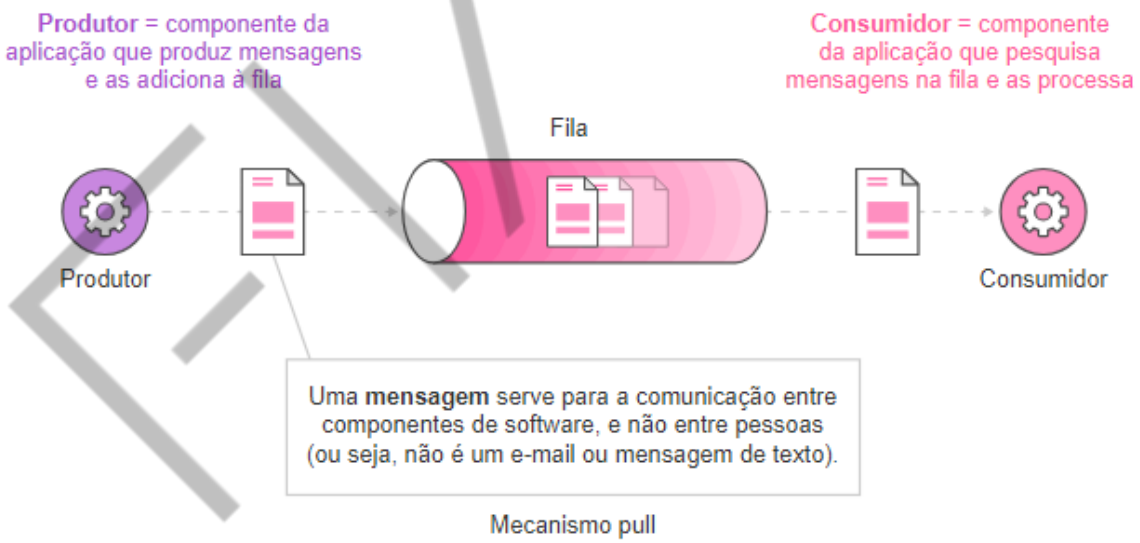

Figura 2 - Fila de Mensagens
Fonte: AWS (2025), adaptado por FIAP (2025)

Na figura anterior, podemos observar o mecanismo de fila, no qual um produtor, como uma aplicação, gera uma mensagem e encaminha a uma fila específica.

As mensagens vão se acumulando em uma fila e podem ser consumidas por outras aplicações. Vale destacar que as aplicações consumidoras precisam consultar ativamente a fila para verificar a existência de mensagens, e, assim, consumi-las.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Esse processo é conhecido como **mecanismo pull**, em que os consumidores acessam constantemente uma fila, leem a mensagem e, em seguida, a removem da fila.

As mensagens no SQS podem conter até 256 KB de dados (incluindo texto e o formato JSON). Elas são mantidas temporariamente na fila até serem processadas por um consumidor.

O SQS pode trabalhar com os seguintes tipos de fila:

- **Fila padrão (Standard Queue):** provê alta escalabilidade, mas sem garantia de ordem exata das mensagens.
- **Fila FIFO:** mantém a ordem exata das mensagens e garante que cada mensagem seja processada apenas uma vez. Ideal para processamento ordenado.

Observe as diferenças entre os tipos de fila:

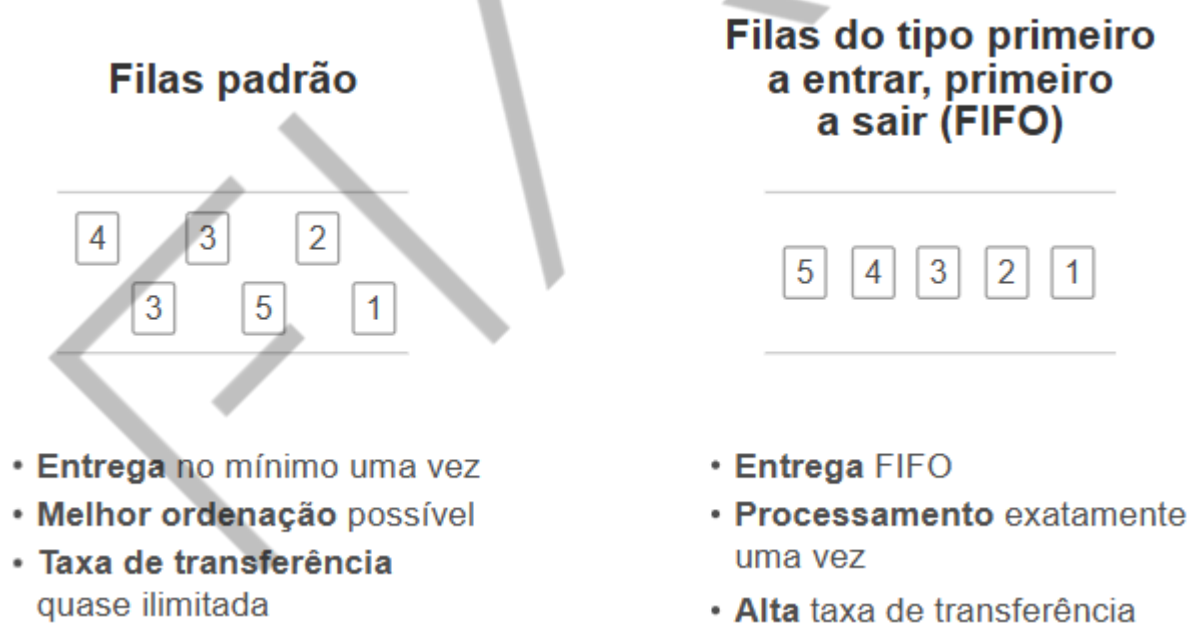

Figura 3 - Tipos de filas do SQS
Fonte: AWS (2025), adaptado por FIAP (2025)

Na figura anterior, podemos comparar os tipos de filas que o SQS consegue trabalhar. Na fila padrão, há uma taxa de transferência muito alta, porém, não há uma garantia de ordenação. Em contrapartida, na fila do tipo FIFO, há a garantia de ordenação, porém, com uma taxa de transferência ligeiramente inferior.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

No tipo de fila FIFO, a ordenação é realizada utilizando o atributo “MessageGroupId” e de duplicação, por meio do atributo “MessageDeduplicationId” definido pelo próprio usuário. Caso uma mensagem com o mesmo “MessageDeduplicationId” seja enviada dentro de um período de 5 minutos, ela será ignorada.

Uma vez que uma mensagem é recebida por um consumidor, ela entra em um período de **timeout** (tempo de visibilidade), impedindo que outros consumidores a processem novamente até que o tempo expire. Se a mensagem não for confirmada como processada, ela retorna à fila para ser processada novamente.

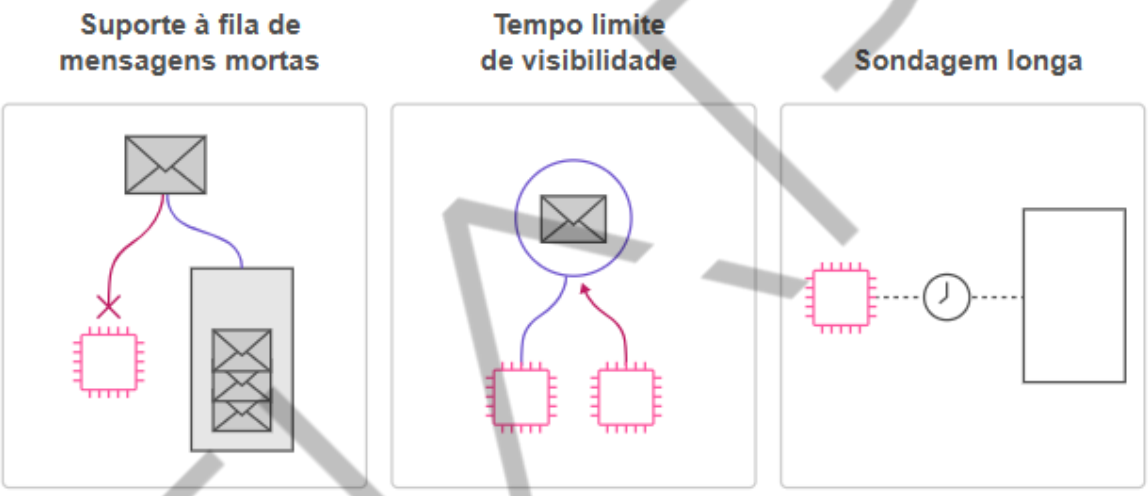

Figura 4 - Dead Letter Queue
Fonte: AWS (2025), adaptado por FIAP (2025)

O SQS pode armazenar mensagens que falharam em uma fila chamada de **Dead Letter Queue (DLQ)**.

É possível configurar a entrega de mensagens das seguintes formas:

- **Sondagem curta (*Short Polling*):** retorna mensagens da fila imediatamente, mesmo que a fila esteja vazia. Pode gerar requisições desnecessárias.
- **Sondagem longa (*Long Polling*):** aguarda até 20 segundos antes de retornar uma resposta, reduzindo custos de requisição e melhorando a eficiência das entregas.

O SQS também suporta processamento em lote (*batch processing*) com o envio e a exclusão de até 10 mensagens por vez.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Antes de iniciarmos nosso projeto, vamos abordar os conceitos básicos de outro serviço de mensageria como o Simple Notification Service (SNS).

2.2 Simple Notification Service (SNS)

O Amazon SNS é um serviço de mensageria do tipo **"pub/sub"** (publicação/assinatura) gerenciado pela AWS, projetado para entrega assíncrona de mensagens.

O SNS permite que publicadores (**publishers**) enviem mensagens para um **tópico SNS** e os **assinantes** (**subscribers**) se inscrevam em tópicos específicos para serem notificados automaticamente.

Um **tópico** no SNS é um canal lógico de comunicação que permite o envio de mensagens para múltiplos assinantes (subscribers). Ele atua como um ***hub***, onde os publicadores enviam mensagens e os assinantes recebem essas mensagens automaticamente.

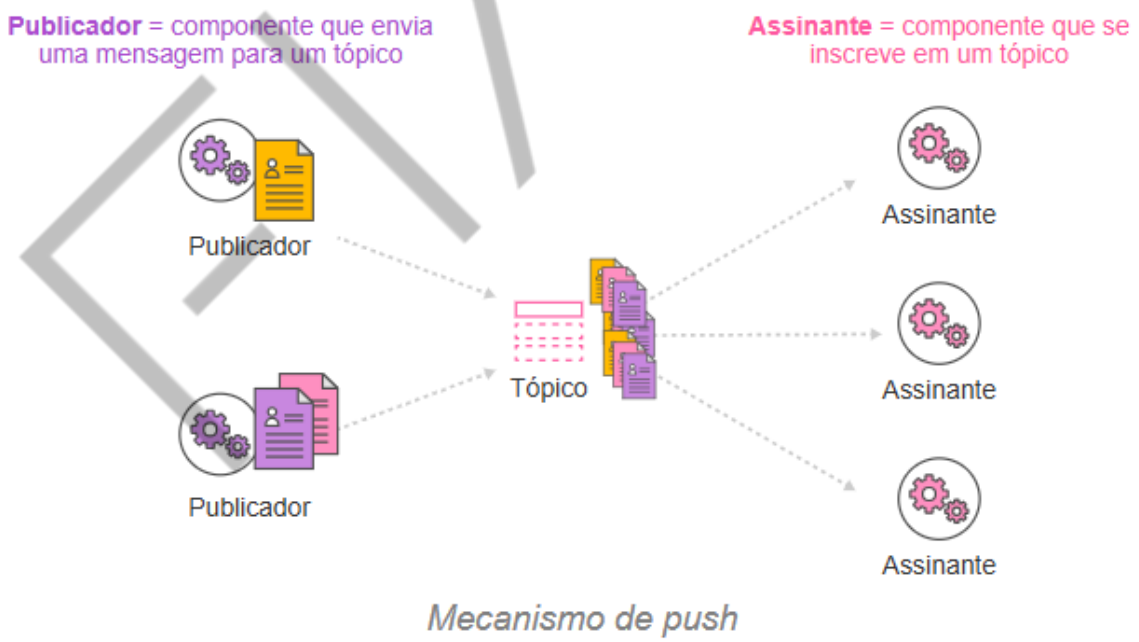

Figura 5 - Mecanismo de *push*
Fonte: AWS (2025), adaptado por FIAP (2025)

O envio de mensagens utiliza o mecanismo de ***push***, em que os assinantes recebem passivamente as mensagens à medida em que são publicadas nos **tópicos**.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Esse mecanismo é bastante utilizado em nosso dia a dia, como no caso de recebimento de notificações em um smartphone como, por exemplo, as mensagens de texto por SMS.

O SNS suporta diversos protocolos e consegue enviar mensagens para serviços da AWS ou serviços externos à AWS, como e-mail e SMS.

Vejamos a seguir uma ilustração desses recursos:

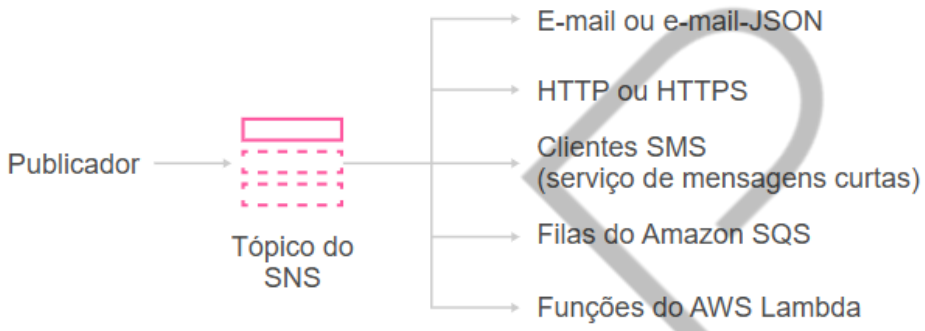

Figura 6 - Protocolos de Transporte SNS
Fonte: AWS (2025), adaptado por FIAP (2025)

Na figura “Protocolos de Transporte SNS”, podemos observar o fluxo de publicação de mensagens para um tópico do SNS. A partir da publicação, a mesma mensagem pode ser enviada para diversos assinantes, como e-mails cadastrados, envio de mensagens SMS, filas do SQS e até mesmo funções Lambda.

Quando uma mensagem é entregue, não há como buscá-la novamente e o SNS não oferece recursos de armazenamento de mensagens.

2.3 SNS *versus* SQS

Diferentemente do SQS, que utiliza o mecanismo de sondagem (*pull*) em que os clientes precisam consultar ativamente uma fila, o SNS envia as mensagens por seu mecanismo de *push*. Ou seja, o SNS consegue enviar uma mensagem para vários assinantes ao mesmo tempo, o que não é possível com o SQS. Entretanto, o SNS não oferece recursos de reprocessamento de mensagens, como o envio para uma fila morta (Dead letter Queue) como acontece no SQS.

Vejamos um resumo das características do SNS em comparação com o SQS no quadro a seguir:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Recurso	SNS (Publicador/Assinante)	SQS (Produtor/Consumidor)
Produtor/consumidor	Publicar/Assinar	Enviar/Receber
Mecanismo de entrega	*Push* (passivo)	Sondagem (ativa)
Modelo de distribuição	Muitos para muitos	Um para um
Persistência de mensagens	Não	Sim

Quadro 1 - SNS x SQS
Fonte: Elaborado pelo autor (2025)

Conforme observado no quadro “SNS x SQS”, o SNS é um serviço do tipo pub/sub em que podemos distribuir mensagens para assinantes ao mesmo tempo.

Pode ser usado em situações como:

- Notificação de múltiplos sistemas em tempo real;
- Envio de eventos para diversas filas do SQS, funções Lambda, e-mail e SMS;
- Uso em arquiteturas orientadas a eventos, como em microsserviços.

Para ilustrar melhor, poderíamos usar o SNS para notificar que um pedido foi aprovado, enviando as mensagens para filas separadas, como a de atualização de estoque e de pagamento, por exemplo. Já o SQS é uma fila de mensagens que garante que cada mensagem seja processada pelo menos uma vez e que pode reter mensagens por 1 minuto até 14 dias, diferentemente do SNS pois ele não permite armazenamento e faz entregas em tempo real.

2.4 Precificação

Ambos os serviços do SQS e do SNS são gratuitos para o primeiro milhão de solicitações (*requests*). Após este valor, há cobranças diferenciadas de acordo com a região da AWS.

Dica: você pode criar uma estimativa com a calculadora de custos da AWS para estimativas atualizadas: <https://calculator.aws/#/>.

A seguir, veremos como utilizar estes serviços de mensageria para viabilizar nosso projeto.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3 CONSTRUINDO NOSSO PROJETO

3.1 Criação de API

Inicialmente, precisaremos configurar uma API que receberá mensagens de pedidos de clientes. Para isso, será necessário criar uma função Lambda para ser acionada no recebimento de novas mensagens por meio de nossa API.

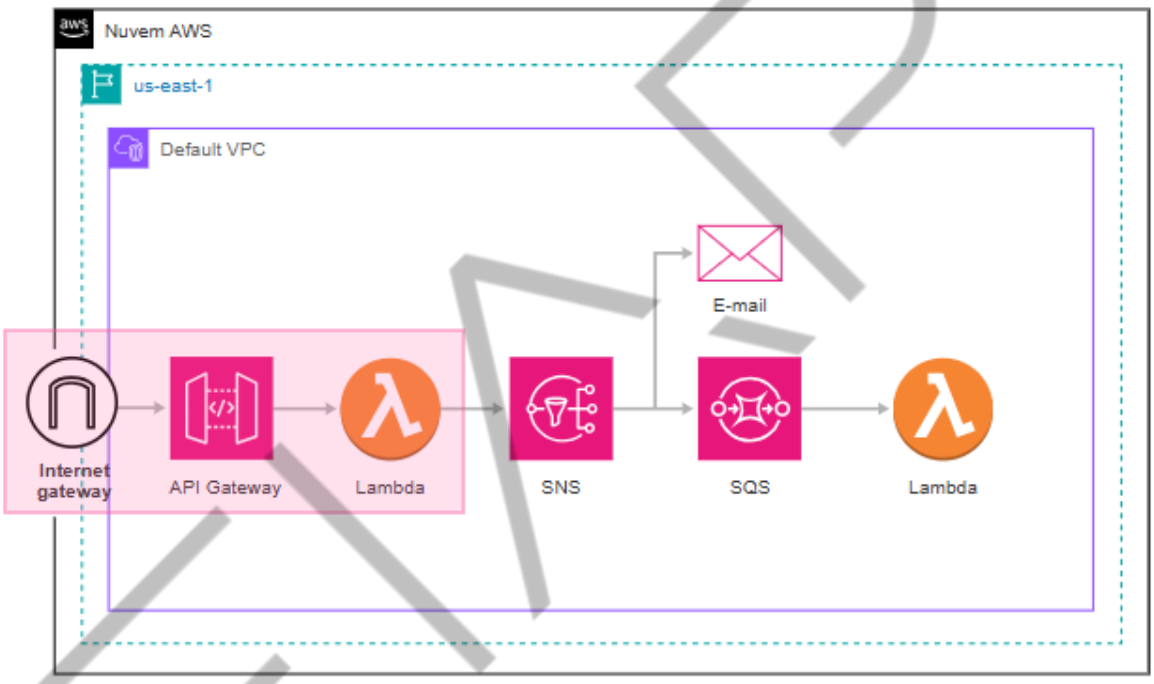

Figura 7 - Definição da API e função Lambda para integração
Fonte: Elaborado pelo autor (2025), adaptado por FIAP (2025)

Conforme destacado na figura “Definição da API e função Lambda para integração”, iremos preparar uma API que será exposta na Internet para receber os dados dos clientes.

Utilizaremos a seguinte função Lambda para processamento de novos pedidos:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
import json
import boto3
import os

def lambda_handler(event, context):
    sns_client  =  boto3.client("sns",  region_name="us-
east-1")
    SNS_TOPIC_ARN = os.getenv("SNS_TOPIC_ARN")

    try:
        if "body" not in event or not event["body"]:
            return    {"statusCode":    400,    "body":
json.dumps({"error": "Nenhum arquivo enviado"})}

        pedido = json.loads(event["body"])
        pedido_id = pedido.get("pedido_id")

        if not pedido_id:
            return    {"statusCode":    400,    "body":
json.dumps({"error": "Campo 'pedido_id' é obrigatório"})}

        # Publica a mensagem no SNS
        response = sns_client.publish(
            TopicArn=SNS_TOPIC_ARN,
            Message=json.dumps(pedido),
            Subject="Novo Pedido"
        )

        return {
            "statusCode": 200,
            "body": json.dumps({
                "message":  "Pedido  recebido  e  enviado
para processamento",
                "pedido_id": pedido_id
            })
        }
    except Exception as e:
        return     {"statusCode":     500,     "body":
json.dumps({"error": str(e)})}
```

Código-fonte 1 - recebe-pedidos-api.py
Fonte: Elaborado pelo autor (2025)

No código anterior, criamos a função Lambda “receber-pedidos-api” que será integrada à nossa API. A função Lambda irá processar o arquivo JSON recebido e após o processamento, enviará uma mensagem a um tópico do SNS com o todo o corpo do pedido processado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Note que para o endereço do tópico, usaremos uma variável de ambiente "`SNS_TOPIC_ARN`" que armazenará o endereço ARN (Amazon Resource Name) do tópico criado.

Enviaremos um pedido utilizando a ferramenta gratuita **Postman** para simular o envio de pedidos. O Postman permite realizar requisições via método POST do protocolo HTTP.

A estrutura das mensagens a serem enviadas está apresentada a seguir:

```
      {
         "pedido_id": "1010",
         "cliente": "Joao Silva",
         "valor": 250.00,
         "itens": [
             {"produto": "Camiseta", "quantidade": 2, "preco":
50.00},
             {"produto": "Bermuda", "quantidade": 1, "preco":
200.00}
         ]
     }
```

Código-fonte 2 - Arquivo JSON de exemplo
Fonte: Elaborado pelo autor (2025)

Na estrutura do pedido, enviaremos os dados do cliente, o valor total do pedido e os itens adquiridos.

Agora, vamos criar a nossa API HTTP no API Gateway chamada api-recebimento-pedidos. Em seguida, criaremos uma rota do tipo POST para o recurso "/pedidos", conforme a figura "Rota para pedidos".

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

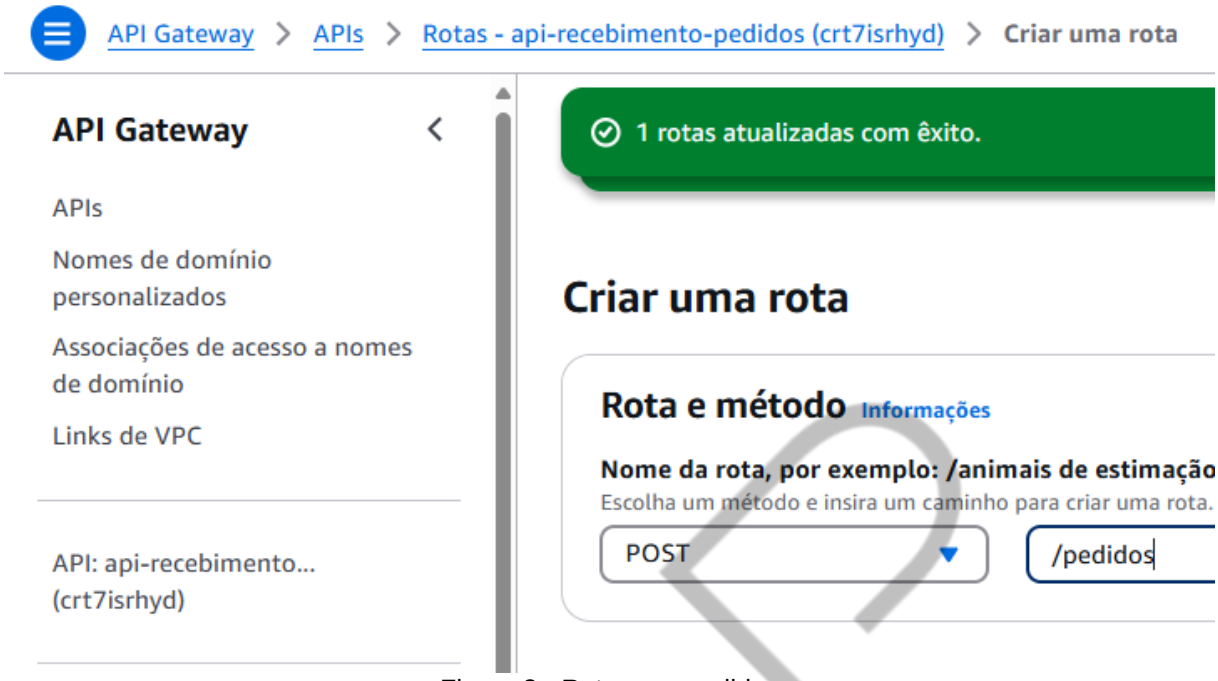

Figura 8 - Rota para pedidos
Fonte: Elaborado pelo autor (2025)

Após a criação da rota, vamos anexar uma integração à função Lambda “recebe-pedidos-api” que criamos anteriormente.

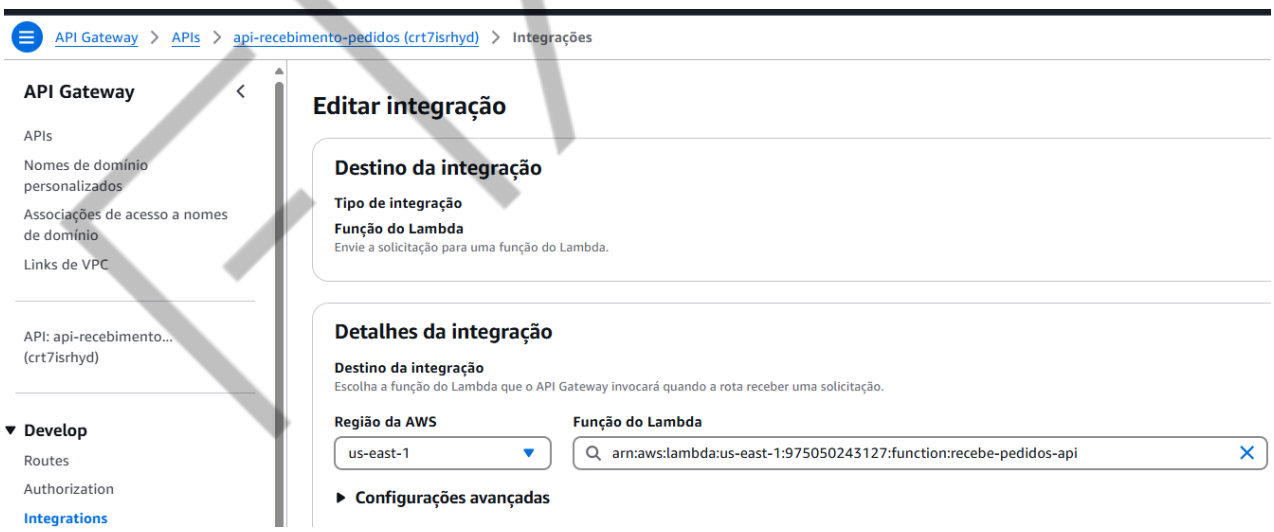

Figura 9 - Integração com função Lambda
Fonte: Elaborado pelo autor (2025)

Na figura “Integração com função Lambda”, podemos observar que foi selecionada a região “us-east-1” da AWS, assim como a função Lambda que criamos anteriormente, denominada “recebe-pedidos-api”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Já temos a nossa API e a função Lambda preparadas para recebimento do pedido. Clique na API e anote o ***endpoint*** padrão que usaremos mais adiante.

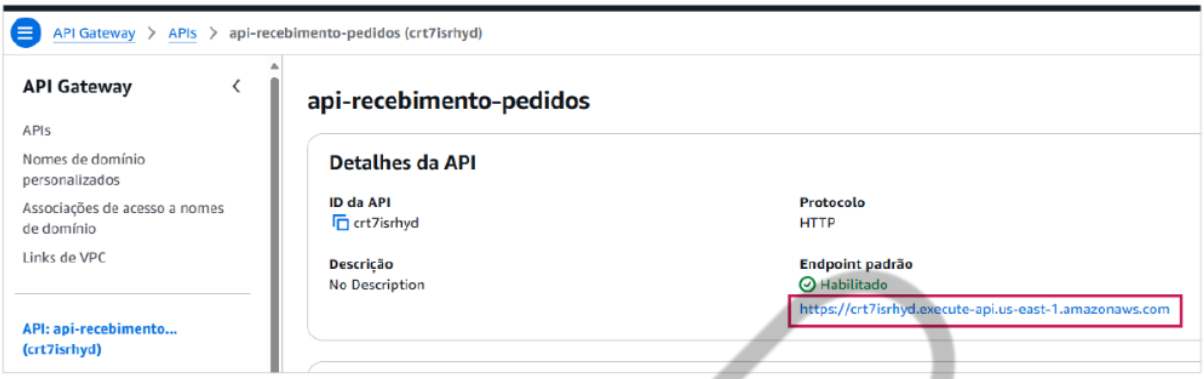

Figura 10 - *Endpoint* da API
Fonte: Elaborado pelo autor (2025)

Verifique a URL para a base de nosso *endpoint*, destacado na figura anterior. Iremos utilizar esta URL em conjunto com o recurso “/pedidos” para realizar inclusões via método POST de novos pedidos.

A figura a seguir ilustra um exemplo de uma chamada via método POST ao nosso *endpoint*.

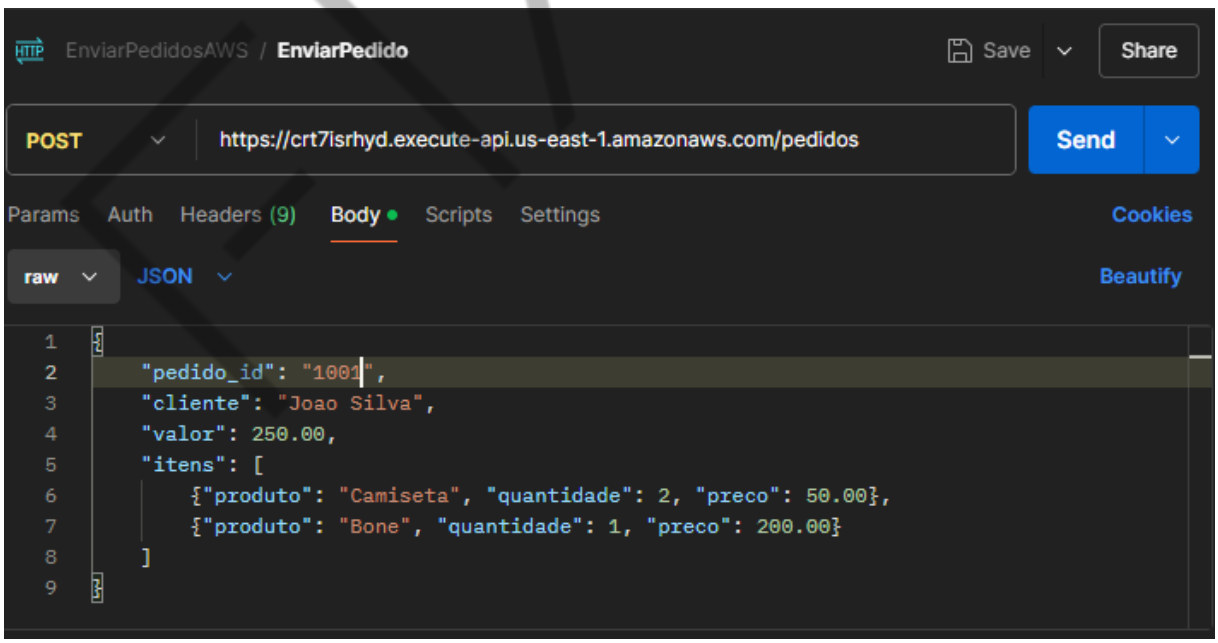

Figura 11 - Exemplo de chamada de API
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Conforme visto na figura “Exemplo de chamada de API”, utilizamos o software Postman que permite realizar chamadas de APIs REST, além de especificar o método HTTP adequado.

Seguiremos agora para a criação do tópico SNS e a criação de uma assinatura para recebimento de confirmação por e-mail.

3.2 Criação do tópico SNS

Nesta etapa, vamos prosseguir com a nossa solução criando um tópico SNS e uma assinatura por e-mail para confirmar o recebimento do pedido.

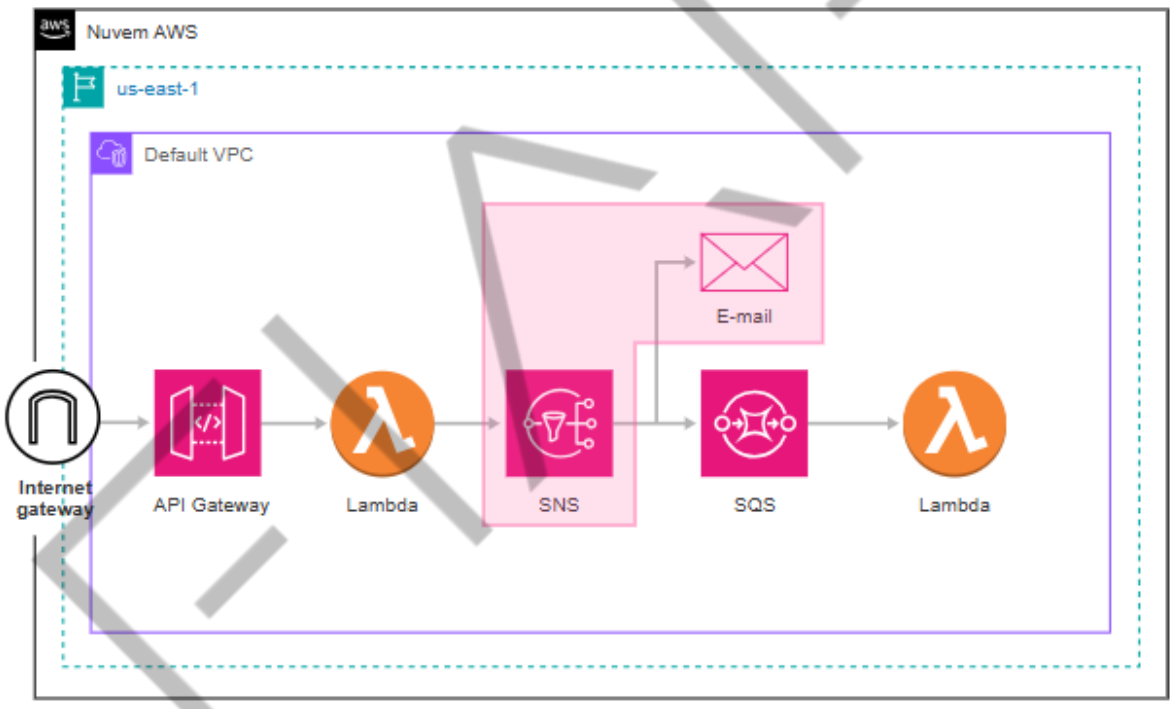

Figura 12 - Criação de tópico e assinatura
Fonte: Elaborado pelo autor (2025), adaptado por FIAP (2025)

Acesse o console de gerenciamento da AWS e busque pelo serviço SNS. Em seguida, clique no menu “Tópicos”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 13 - Menu Tópicos – SNS
Fonte: Elaborado pelo autor (2025)

Escolha o tipo de tópico “Padrão” e nomeie o tópico para “pedido-recebido-topico”. Mantenha o mesmo nome para exibição.

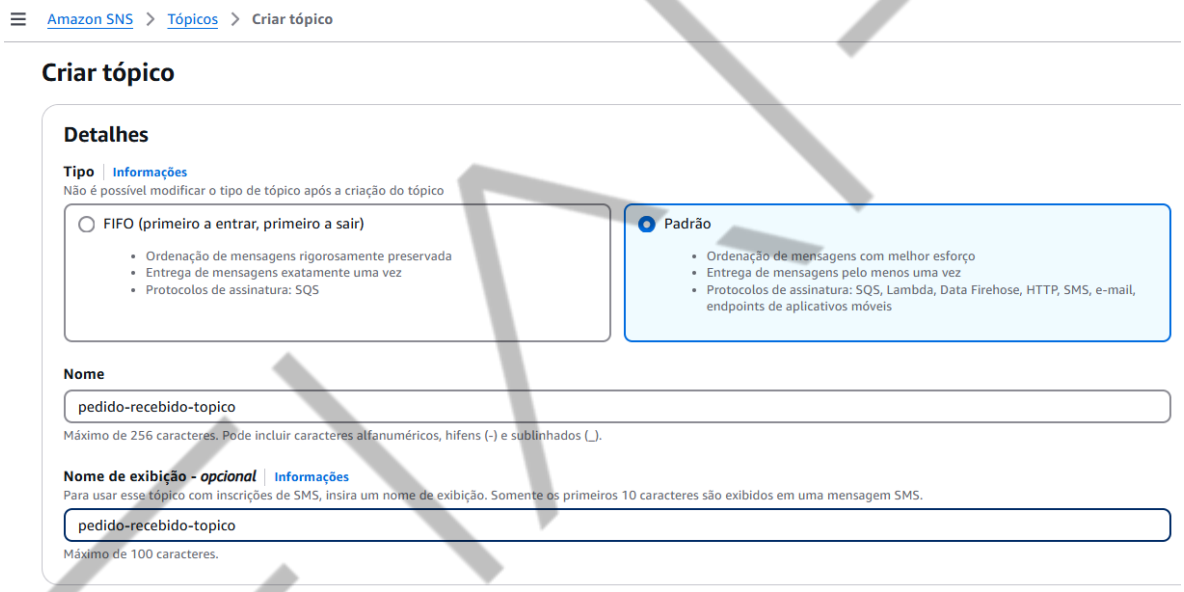

Figura 14 - Criação de tópico pedido-recebido-topico3
Fonte: Elaborado pelo autor (2025)

Após a criação, o tópico será listado no menu tópicos. Anote o ARN do tópico para utilizarmos em nossa função Lambda da API.

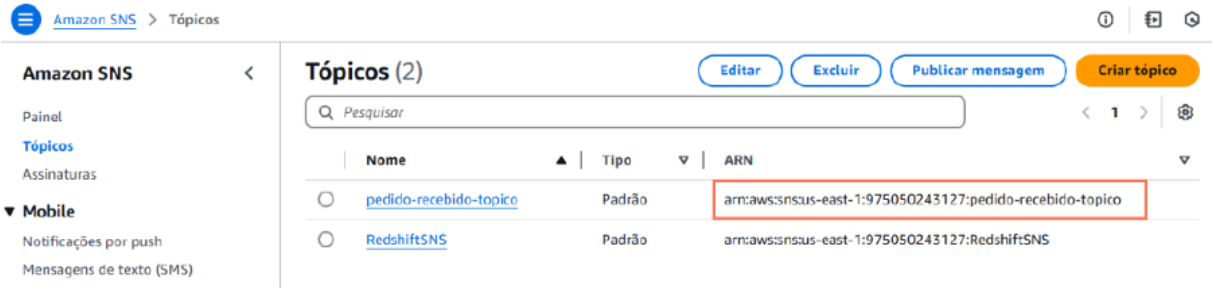

Figura 15 - Tópico criado
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Na figura anterior, podemos observar o tópico criado e o endereço do tópico na infraestrutura da Amazon (ARN). Acesse a função Lambda “recebe-pedidos-api” e localize a opção “ENVIRONMENT VARIABLES”, em seguida clique no botão de adicionar ao lado esquerdo do código da função Lambda.

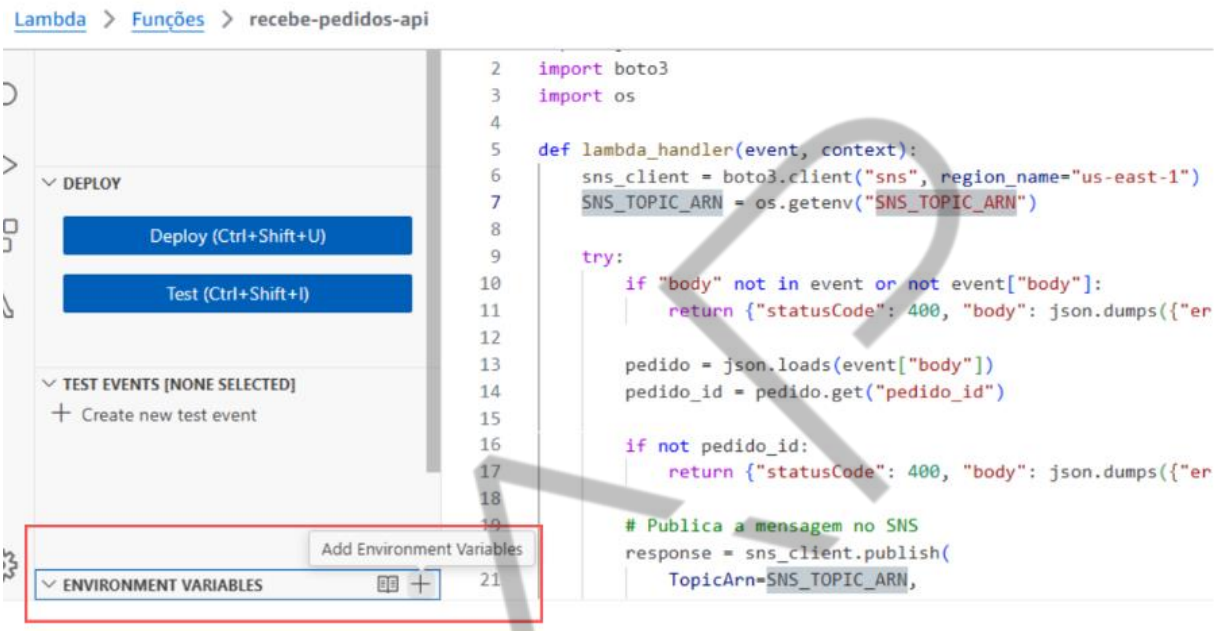

Figura 16 - Criação de variável de ambiente no Lambda
Fonte: Elaborado pelo autor (2025)

Crie uma variável de ambiente que aponte para este endereço:

Figura 17 - Variável de ambiente para o SNS
Fonte: Elaborado pelo autor (2025)

Importante: não se esqueça de salvar as alterações.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Agora iremos criar uma assinatura para um e-mail no tópico SNS “pedido-recebimento-topico”. Acesse o SNS e clique em “Assinaturas”:

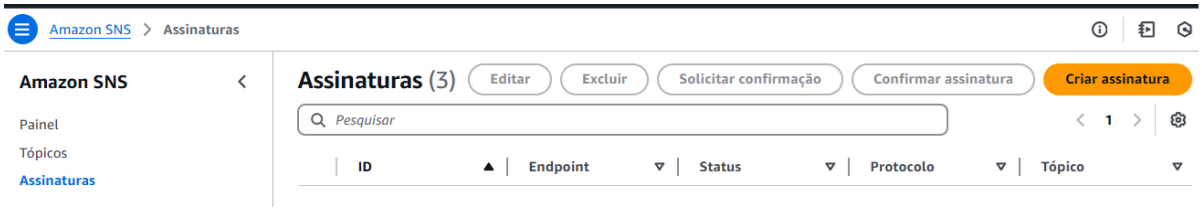

Figura 18 - Visão Geral de Assinaturas no SNS
Fonte: Elaborado pelo autor (2025)

Escolha o tópico no ARN do tópico. Depois escolha o protocolo “E-mail” e digite seu e-mail. Em seguida, clique em “Criar assinatura”.

Figura 19 - Criação de assinatura para e-mail
Fonte: Elaborado pelo autor (2025)

Você receberá um e-mail de confirmação de assinatura. Confirme a criação da assinatura em seu e-mail. Em seguida, a assinatura aparecerá listada.

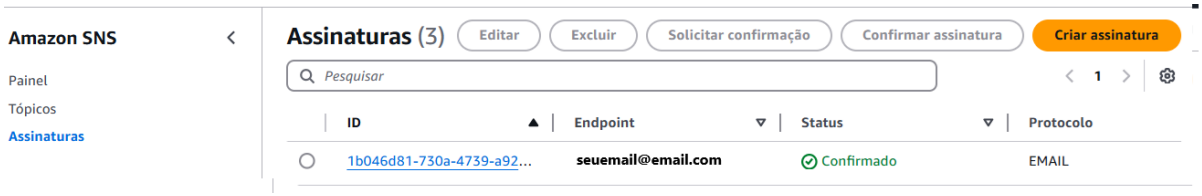

Figura 20 - Assinatura criada
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Pronto! Agora basta enviar uma mensagem por uma aplicação cliente para o *endpoint* da API para testar o recebimento de mensagens pelo SNS. Vejamos o envio de mensagem pelo aplicativo Postman.

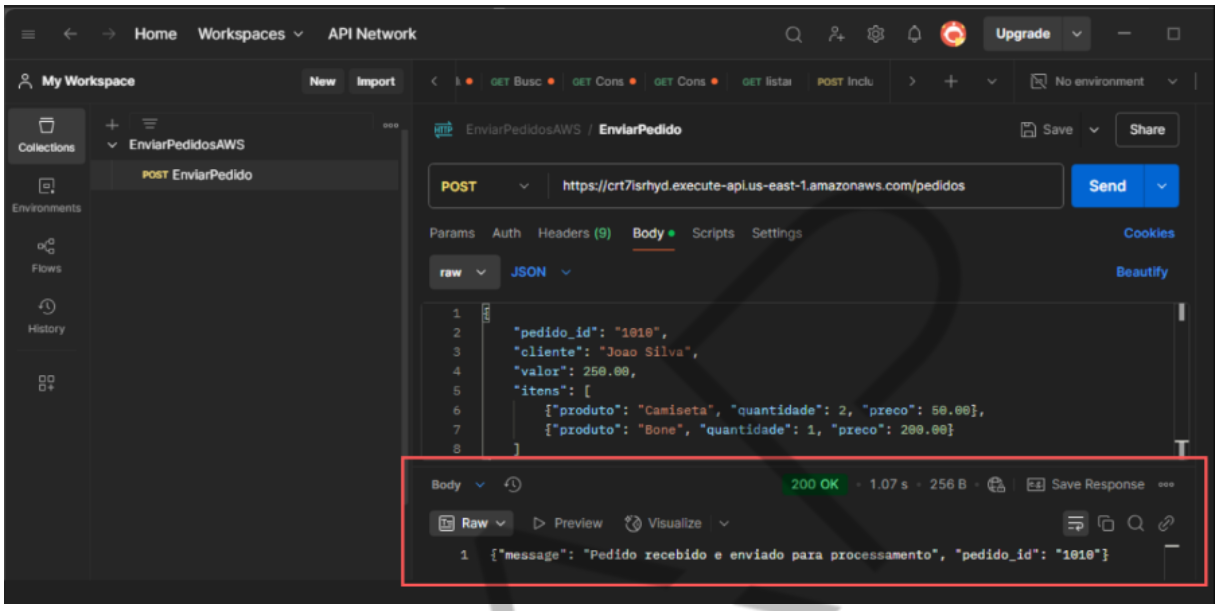

Figura 21 - Envio de mensagens via Postman
Fonte: Elaborado pelo autor (2025)

Note na figura “Envio de mensagens via Postman” o resultado recebido após o envio da solicitação via método POST. O status “200” informa que a solicitação foi recebida com sucesso junto com a mensagem que configuramos de retorno.

Você também receberá um e-mail com o corpo do JSON enviado.

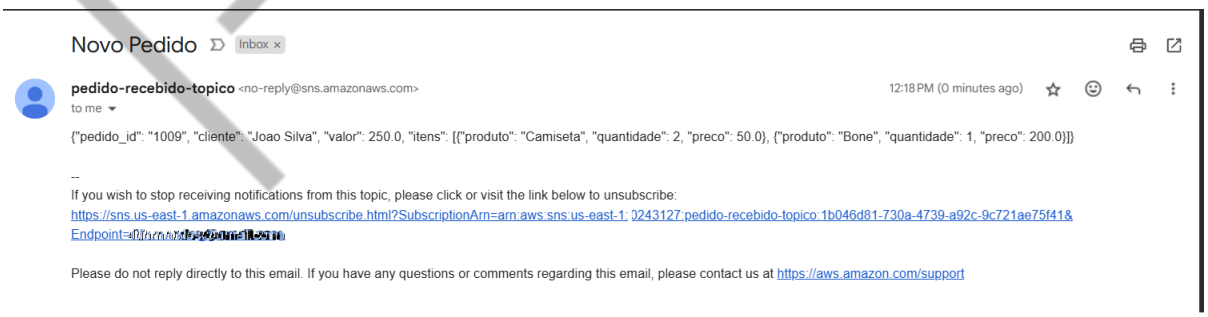

Figura 22 - E-mail recebido
Fonte: Elaborado pelo autor (2025)

Na figura anterior exibimos um exemplo de e-mail recebido, mascarando informações da conta do usuário.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Além do conteúdo JSON, há um link para cancelamento da inscrição, caso o assinante deseje cancelar sua assinatura.

Pronto! Já temos um assinante cadastrado e a esteira concluída desde o recebimento do pedido até o recebimento do e-mail.

Agora, vamos criar a fila SQS para confirmar o pagamento dos pedidos.

3.3 Criação de pilha SQS

Para dar sequência em nosso projeto, iremos criar uma pilha no SQS que será responsável por processar pagamentos dos pedidos.

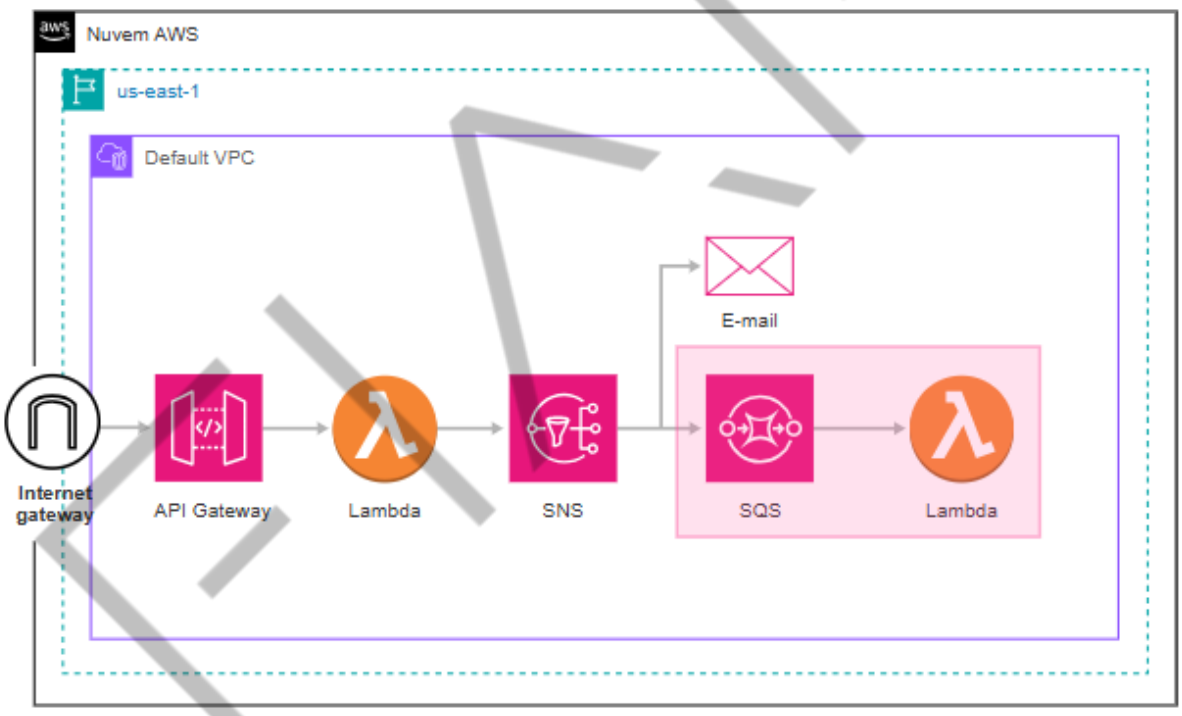

Figura 23 - Criação de pilha SQS
Fonte: Elaborado pelo autor (2025), adaptado por FIAP (2025)

Acesse o serviço SQS e clique em criar pilha.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 24 - Criação de Pilha SQS
Fonte: Elaborado pelo autor (2025)

Defina o tipo de fila como padrão e o nome da fila como “fila-pagamento”.

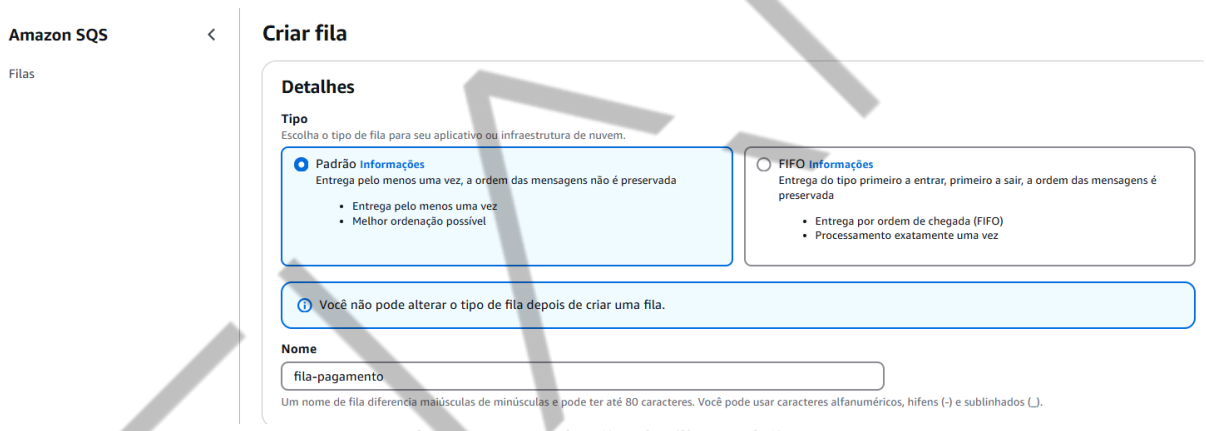

Figura 25 - Criação de fila padrão
Fonte: Elaborado pelo autor (2025)

E lembre-se de adicionar na política de acesso, a conta do SNS, para permitir envio de mensagens à fila.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

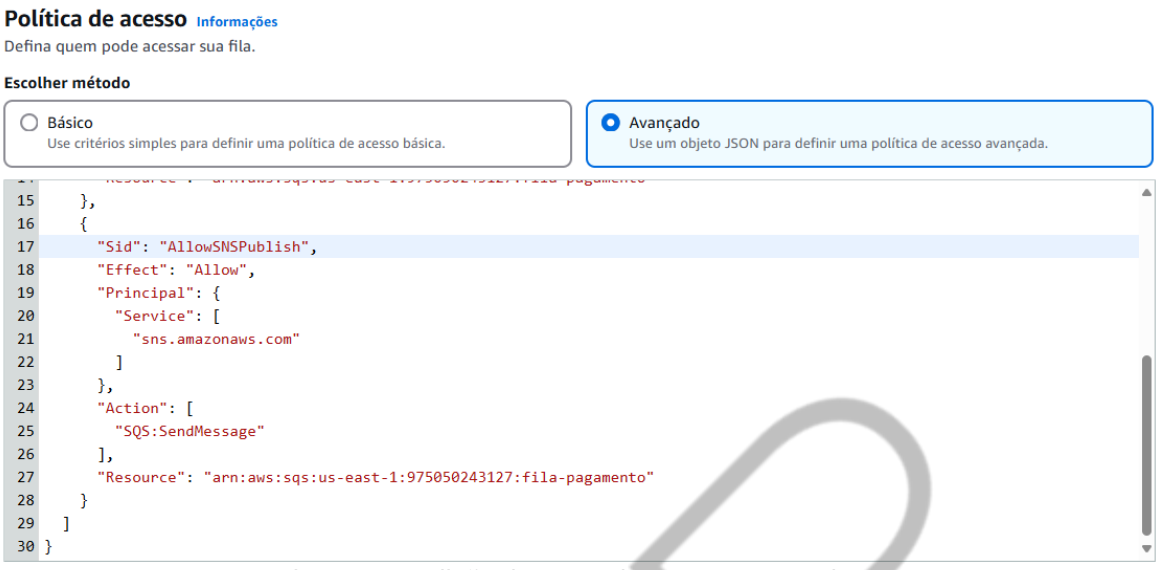

Figura 26 - Adição de conta do SNS como produtor
Fonte: Elaborado pelo autor (2025)

Note que na figura “Adição de conta do SNS como produtor” adicionamos o serviço SNS para envio de mensagens à fila. A seguir, apresentamos o código do JSON completo.

```
      {
        "Version": "2012-10-17",
        "Id": "__default_policy_ID",
        "Statement": [
          {
            "Sid": "__owner_statement",
            "Effect": "Allow",
            "Principal": {
              "AWS": "arn:aws:iam::975050243127:root"
            },
            "Action": "SQS:*",
            "Resource":                    "arn:aws:sqs:us-east-
1:975050243127:fila-pagamento"
          },
          {
            "Sid": "AllowSNSPublish",
            "Effect": "Allow",
            "Principal": {
              "Service": "sns.amazonaws.com"
            },
            "Action": "SQS:SendMessage",
            "Resource":                    "arn:aws:sqs:us-east-
1:975050243127:fila-pagamento"
          }
        ]
      }
```

Código-fonte 3 - Política de acesso da fila SQS
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Agora que temos nossa fila criada, criaremos uma função Lambda que será disparada assim que uma mensagem entrar em nossa fila.

Acesse o serviço do Lambda para criar uma nova função chamada “processar-pagamento”.

```
import json

def lambda_handler(event, context):
    for record in event['Records']:
        mensagem = json.loads(record['body'])
        pedido_id = mensagem['pedido_id']
        print(f"Processando pagamento para o pedido {pedido_id}")
        # Simular processamento de pagamento
        return {"status": "Pagamento processado", "pedido_id": pedido_id}
```

Código-fonte 4 - processar-pagamento.py
Fonte: Elaborado pelo autor (2025)

Adicione um gatilho para a nossa fila SQS de pagamento.

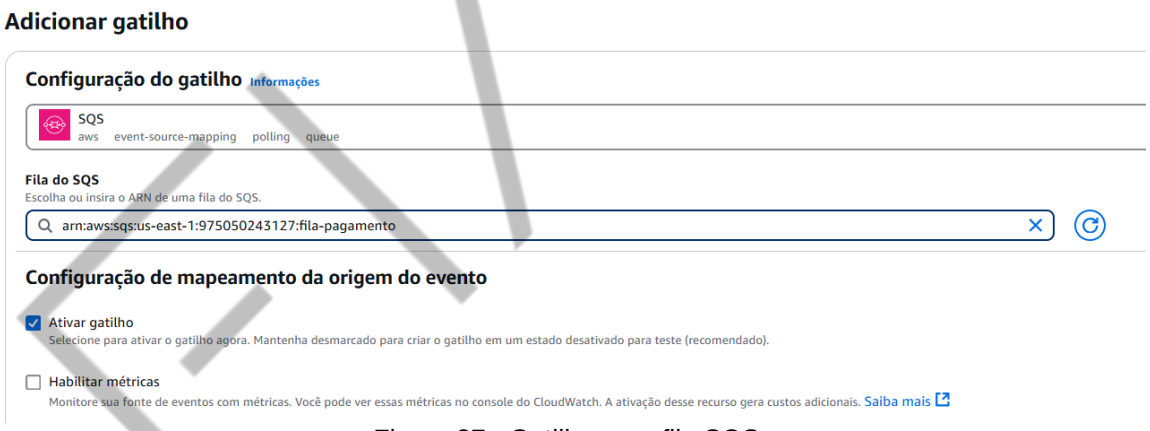

Figura 27 - Gatilho para fila SQS
Fonte: Elaborado pelo autor (2025)

A seguir apresentamos como deve ficar nossa função Lambda.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

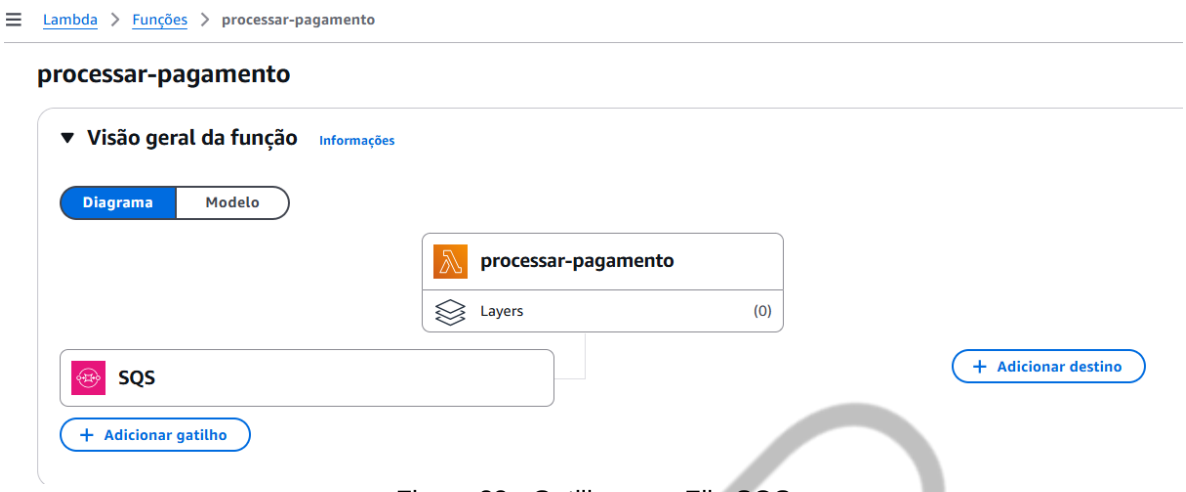

Figura 28 - Gatilho para Fila SQS
Fonte: Elaborado pelo autor (2025)

Pronto! Agora que já temos um gatilho criado, vamos testar nossa esteira de serviços assíncrona.

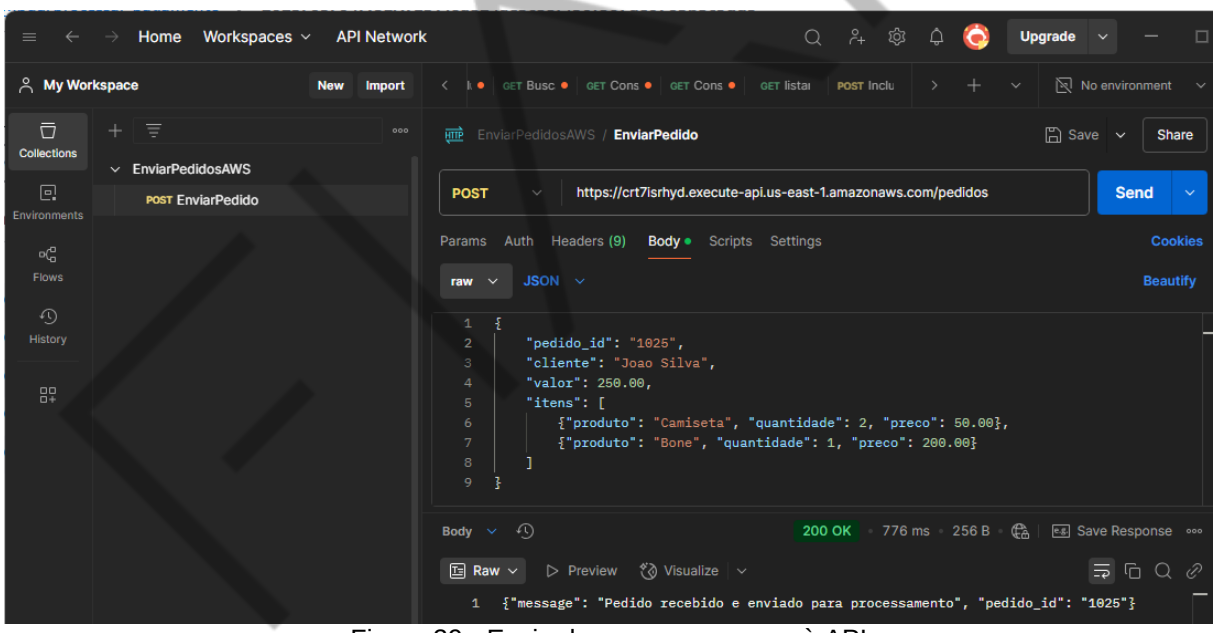

Figura 29 - Envio de nova mensagem à API
Fonte: Elaborado pelo autor (2025)

Como visto na figura anterior, enviamos uma nova mensagem à nossa API. Você deverá receber um novo e-mail com o pedido. Nas próximas etapas, veremos como validar se o pagamento foi devidamente processado pela função Lambda.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.4 Amazon CloudWatch

O Amazon CloudWatch é um serviço da AWS para monitoramento de aplicações, infraestrutura e serviços. Ele coleta, armazena e analisa métricas, logs e eventos, permitindo a criação de alertas, dashboards e automação.

É possível criar métricas em tempo real sobre o uso de recursos, como por exemplo, o uso de CPU em uma instância do EC2 e eventos do Lambda.

Além disso, ele possui logs de aplicações e infraestrutura na AWS, podendo gerar alertas com base nestes logs.

Vamos aprender como acessar o CloudWatch para avaliarmos se nossa função Lambda foi criada com sucesso.

Primeiramente, acesse CloudWatch na barra de pesquisas e clique em “Grupos de logs” no menu lateral.

Figura 30 - Dashboard do CloudWatch
Fonte: Elaborado pelo autor (2025)

Você poderá visualizar todos os grupos de logs criados.

Os grupos de logs podem ser criados automaticamente quando criamos algum serviço monitorável, como uma função Lambda, mas também existe a opção de criarmos manualmente um grupo de log específico.

Clique no grupo de logs referente à nossa função Lambda “/aws/lambda/processar-pagamento”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

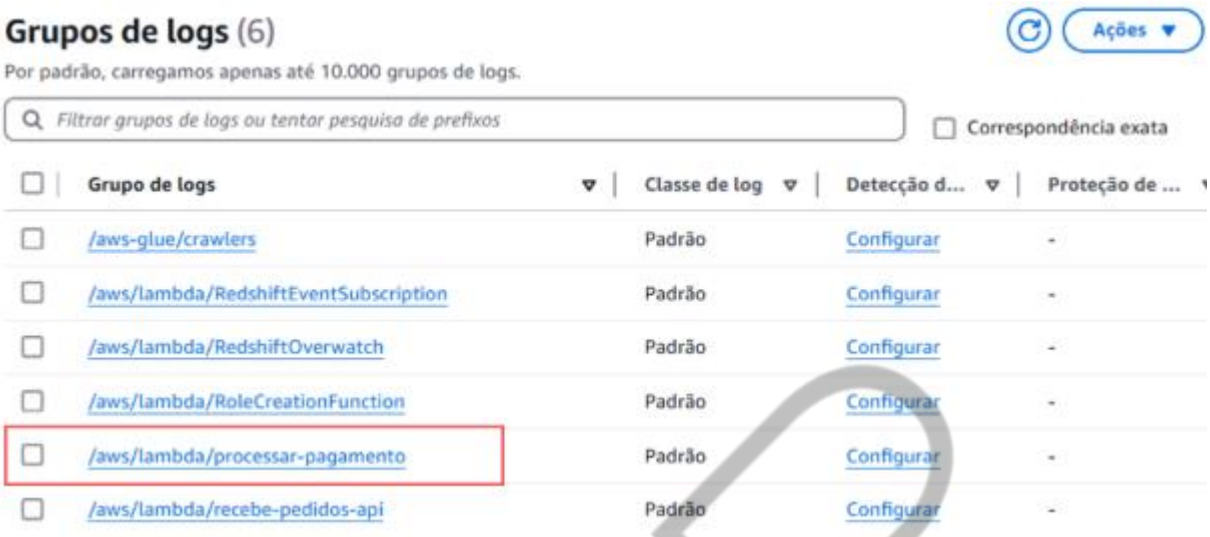

Figura 31 - Lista de Grupos de Logs
Fonte: Elaborado pelo autor (2025)

Você poderá visualizar os logs gerados de acordo com a hora de ocorrência do evento. Clique no link do fluxo de logs referente ao último horário para verificar se sua função Lambda foi executada com sucesso para o pedido enviado.

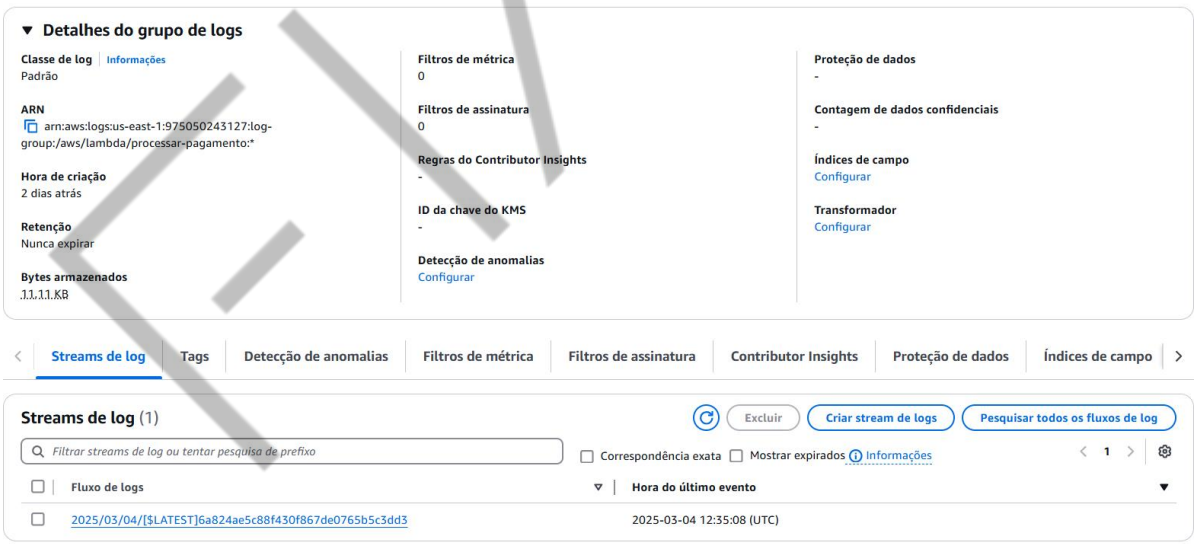

Figura 32 - Log gerado no CloudWatch
Fonte: Elaborado pelo autor (2025)

Você será direcionado para o evento gerado a partir da função Lambda.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 33 - Evento de Log
Fonte: Elaborado pelo autor (2025)

É possível expandir um evento de log para visualizar detalhes do evento.

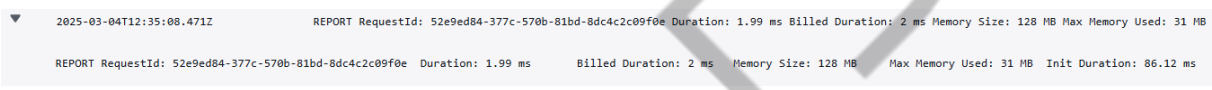

Figura 34 - Detalhes para cobrança da função Lambda
Fonte: Elaborado pelo autor (2025)

Enfim, podemos verificar se a função Lambda executou nosso pedido com sucesso, expandindo os eventos de log.

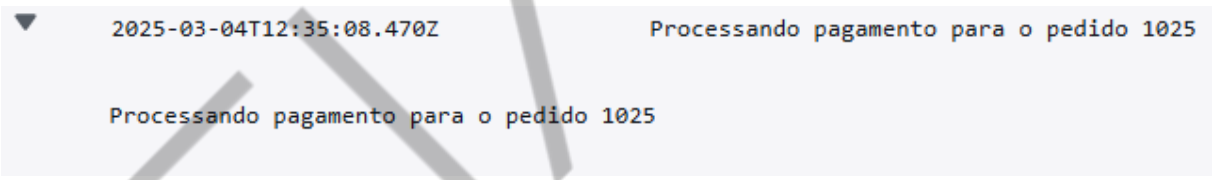

Figura 35 - Evento de processamento concluído
Fonte: Elaborado pelo autor (2025)

Os grupos de log são muito úteis quando trabalhamos com funções Lambda, pois é possível observar se a função foi executada com sucesso ou se houve alguma falha. Quando ocorrem falhas, podemos expandir os eventos de log para tentar identificar a causa raiz do problema e corrigir nossa função Lambda.

Finalmente podemos dizer que concluímos nosso projeto de uma esteira automatizada de processamento de pedidos na AWS!

Agora, para garantir o bom funcionamento da função Lambda, iremos abordar como monitorar as chamadas à função Lambda.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.5 Monitoramento com CloudWatch

Acesse o CloudWatch e clique no menu Métricas.

Figura 36 - Métricas do CloudWatch
Fonte: Elaborado pelo autor (2025)

O CloudWatch irá listar diversos serviços da AWS, incluindo os eventos das funções Lambda. Certifique-se de que esteja na região certa e clique na função Lambda em “Visualizar o painel automático”.

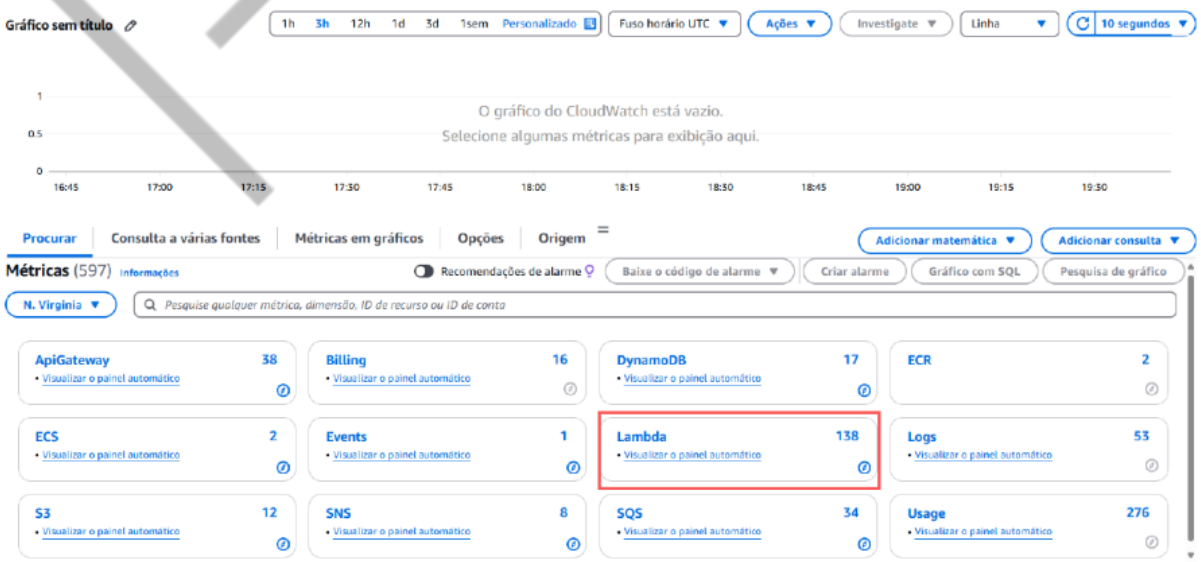

Figura 37 - Dashboard de Métricas
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Será apresentado um dashboard com diferentes métricas, como invocações totais de funções Lambda, total de erros, tempo médio de duração e o número de solicitações de invocação que são limitadas (Throttles).

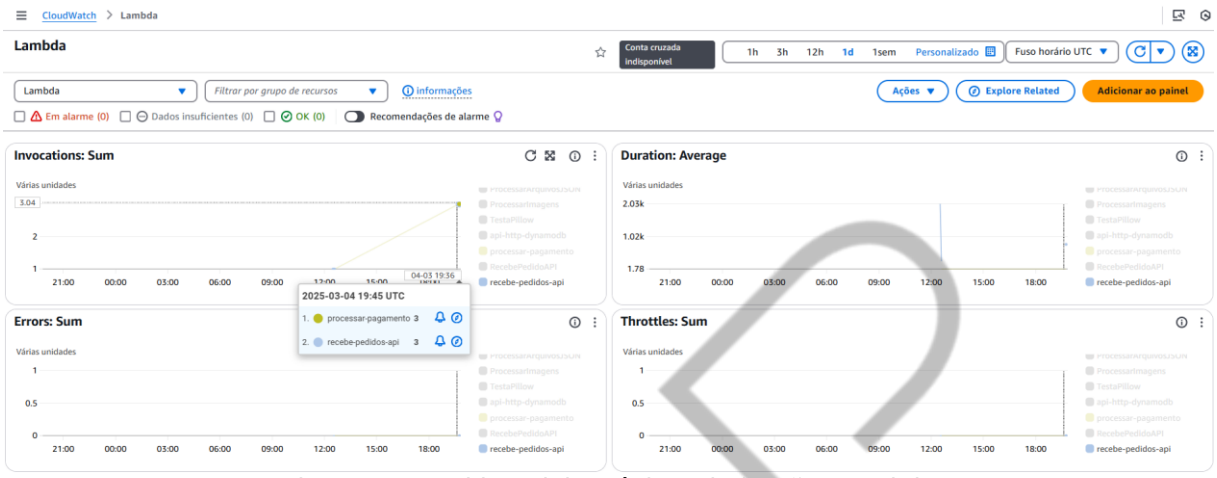

Figura 38 - Dashboard de métricas de funções Lambda
Fonte: Elaborado pelo autor (2025)

Você pode clicar em um elemento do gráfico para ser redirecionado para a criação de alarmes específicos para a métrica selecionada.

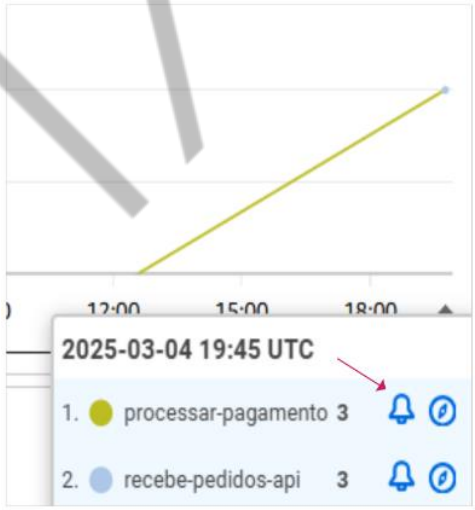

Figura 39 - Criação de alarmes de invocações
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

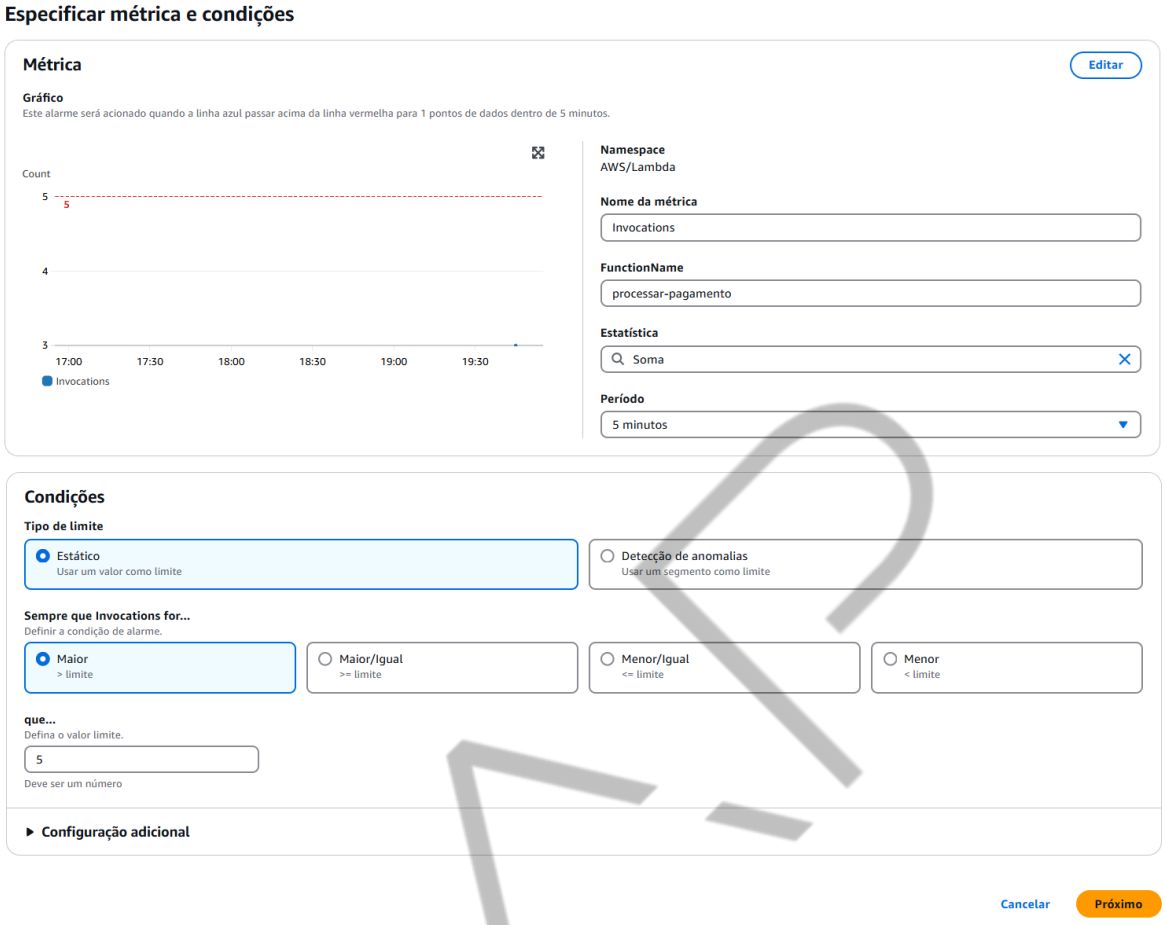

Figura 40 - Condições do alarme
Fonte: Elaborado pelo autor (2025)

Na figura “Condições do alarme”, configuramos o alarme para disparar caso o número de solicitações seja maior do que cinco no período de cinco minutos. Também poderíamos criar alarmes para uma demanda inferior, como a inexistência de pedidos durante 8 horas.

Clique no botão próximo, vamos criar um tópico no SNS para enviar um e-mail assim que o alarme for disparado.

Defina o nome do alarme para “alarme-pedidos”, defina seu e-mail e clique em “Criar tópico”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

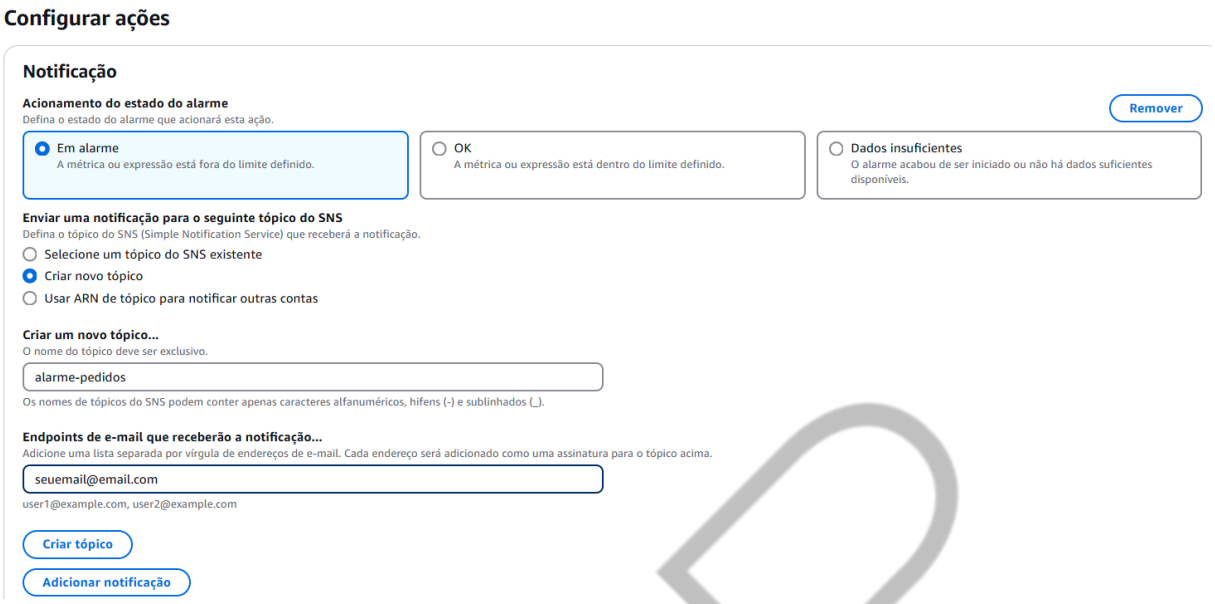

Figura 41 - Criação de tópico para alarme
Fonte: Elaborado pelo autor (2025)

Clique em próximo e defina um nome para o alarme. Você poderá definir também uma mensagem padronizada para o alarme.

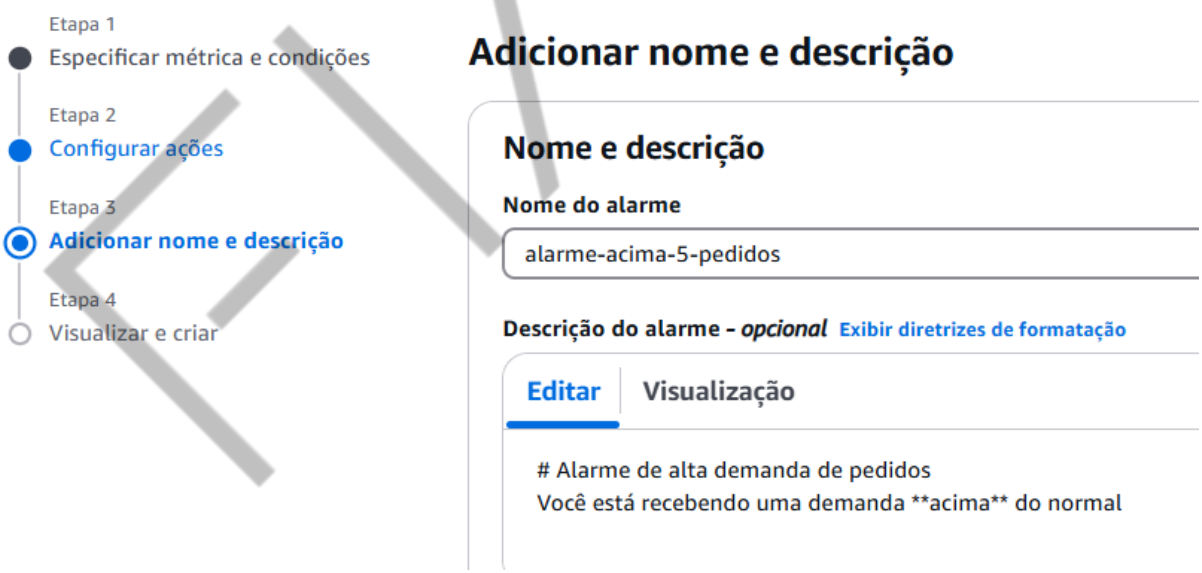

Figura 42 - Definições do alarme
Fonte: Elaborado pelo autor (2025)

Clique em próximo, visualize os detalhes do alarme e clique em criar. Simule o envio de seis pedidos pelo Postman. Aguarde alguns instantes e veja o alarme disparado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

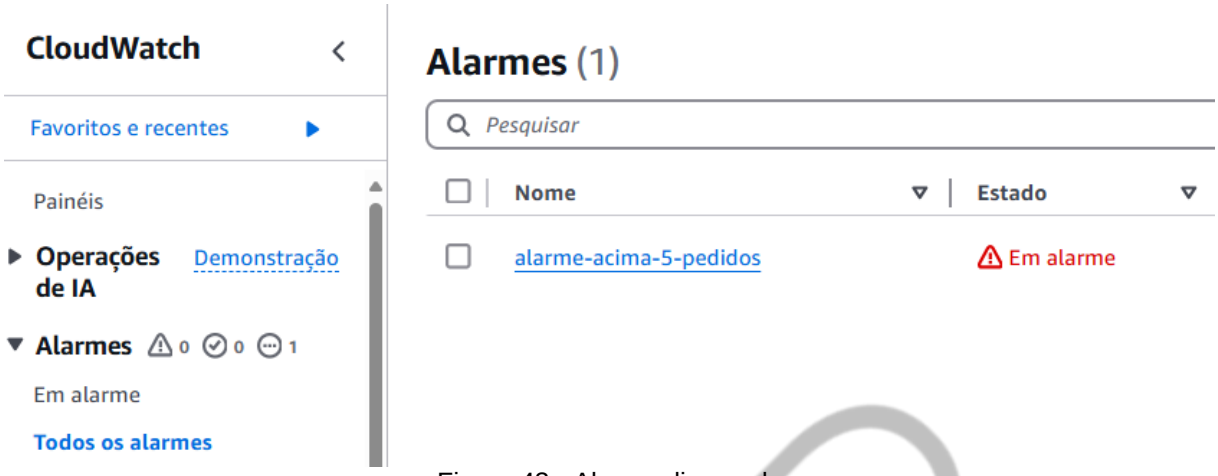

Figura 43 - Alarme disparado
Fonte: Elaborado pelo autor (2025)

Você também receberá no e-mail um aviso sobre o disparo do alarme.

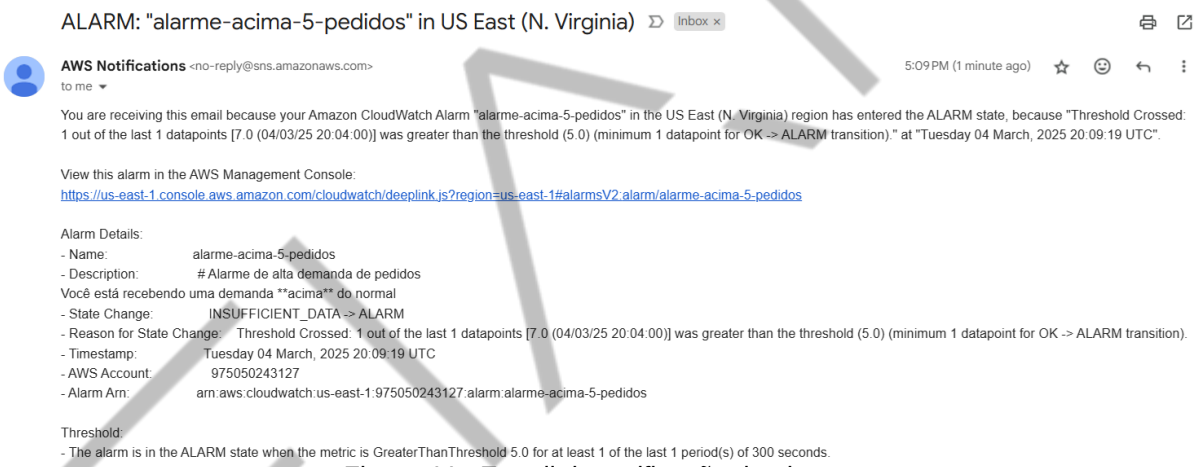

Figura 44 - E-mail de notificação de alarme
Fonte: Elaborado pelo autor (2025)

Agora já temos uma solução escalável e monitorável pronta para uso!

A próxima etapa é criar mais um alarme para disparar em casos de erros da execução da função Lambda.

Acesse o menu de Alarmes e clique em “Todos os alarmes”:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

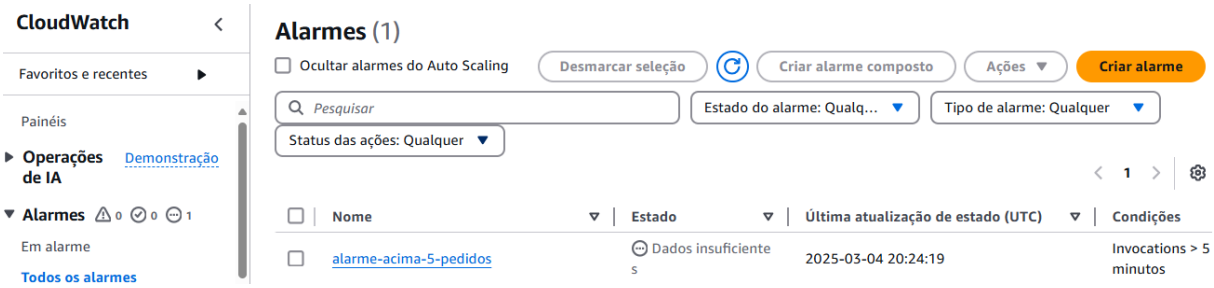

Figura 45 - Criação de alarme de Erro
Fonte: Elaborado pelo autor (2025)

Clique em “Criar alarme”. Em seguida, clique em “Selecionar métrica”.

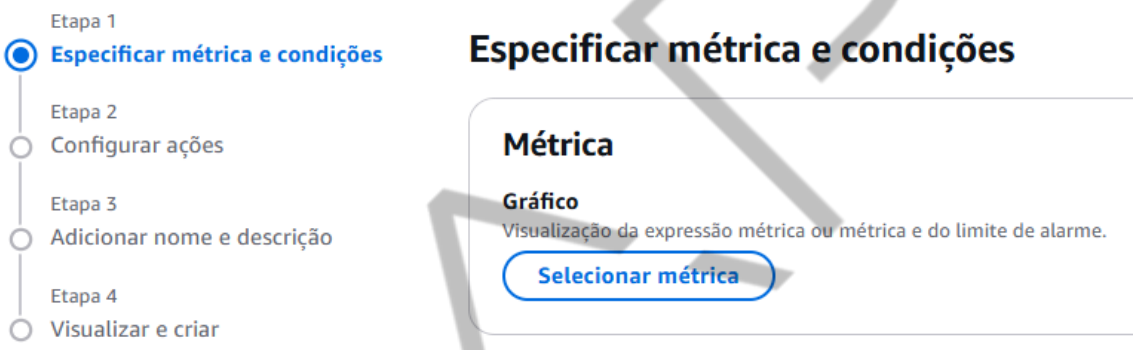

Figura 46 - Seleção de métrica de alarme
Fonte: Elaborado pelo autor (2025)

Selecione “Função Lambda”, em seguida clique em “By Function Name”.

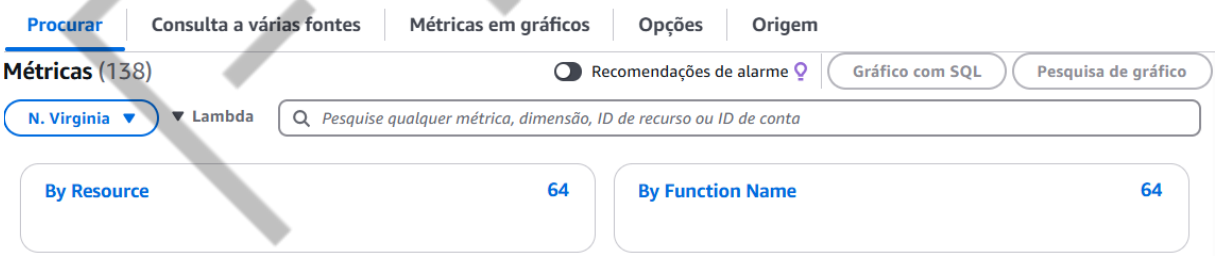

Figura 47 - Seleção de métrica da função Lambda
Fonte: Elaborado pelo autor (2025)

Escolha a função “processar-pagamento” e a métrica “errors”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

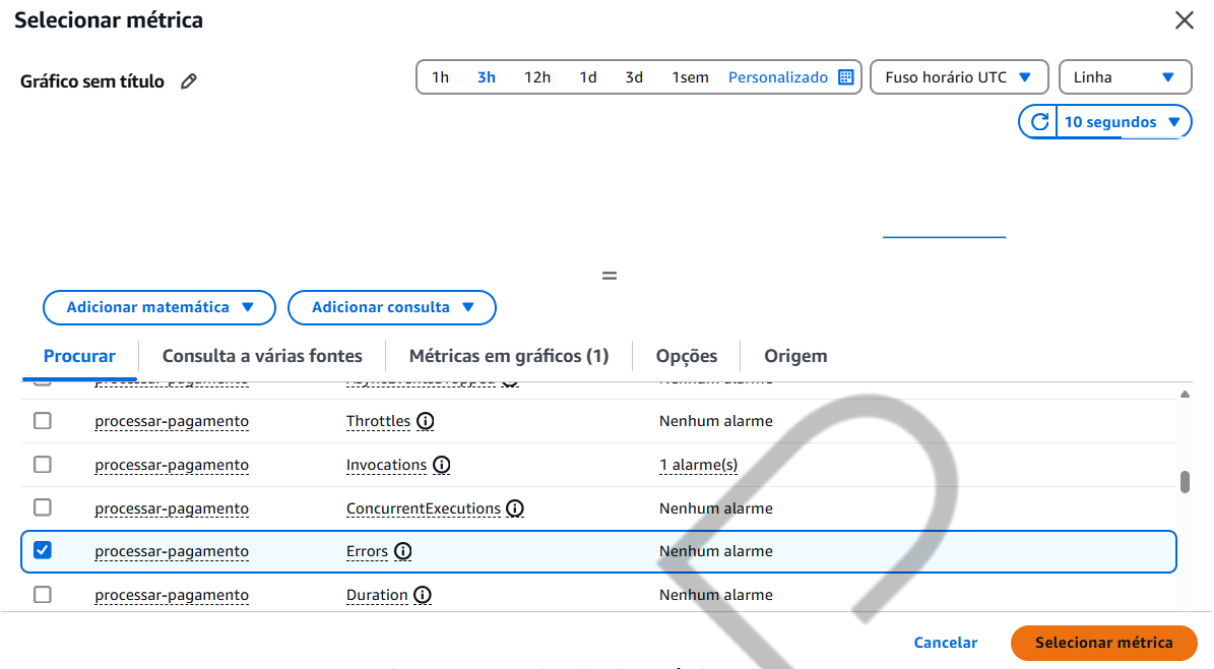

Figura 48 - Criação de métrica de erros
Fonte: Elaborado pelo autor (2025)

Escolha a opção de maior que 1, crie um novo tópico no SNS “erro-processar-pagamento” e escolha o envio para o seu e-mail.

Defina um nome para o alarme e crie o alarme. Pronto! Caso ocorra um erro no processamento, você receberá uma mensagem em seu e-mail.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

CONCLUSÃO

A combinação de serviços de mensageria como Amazon SNS e o Amazon SQS, juntamente com funções Lambda, cria uma **arquitetura altamente escalável**, desacoplada e resiliente para processamento de pedidos na AWS.

Aprendemos como, a partir de um *endpoint* exposto por uma API Lambda, podemos receber pedidos de clientes em um formato universal (JSON) e encaminhar os pedidos para um tópico do SNS. Ao receber mensagens em seus tópicos, o SNS distribui essas mensagens para todos os seus assinantes, como vimos no envio de mensagens por e-mail e o envio de mensagens a uma fila SQS.

O SQS por sua vez, a partir do recebimento da mensagem em sua fila, permite disparar um gatilho a uma função Lambda para processar o pagamento do pedido.

E, por fim, aprendemos como monitorar serviços na AWS por meio do CloudWatch, que gera logs que facilitam a depuração de uma arquitetura distribuída na nuvem.

Essa abordagem permite **automação total**, melhor tolerância a falhas e maior eficiência no processamento de eventos, encadeando eventos em uma arquitetura distribuída.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

ARQUITETURA Medalhão. **DataBricks**, 2025. Disponível em: <https://www.databricks.com/br/glossary/medallion-architecture>. Acesso em: 13 mar. 2025.

AWS Academy. **AWS Academy**, 2025. Disponível em: <https://awsacademy.instructure.com/>. Acesso em: 13 mar. 2025.

CULKIN, J; ZAZON, M. **AWS Cookbook. Recipes for Success on AWS.** O' Reilly, 2022.

EAGAR, G. **Data Engineering with AWS.** Birmingham, UK: Packt Publishing, 2021.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados.** 7º Ed.; São Paulo: Pearson Education do Brasil, 2018.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

GLOSSÁRIO

AWS	Amazon Web Services.
CLI	Command Line Interface.
EBS	Elastic Block Storage.
EC2	Elastic Compute Cloud.
EFS	Elastic File System.
RDS	Relational Database Service.
ELT	Extração, Carga e Transformação.
ETL	Extração, Transformação e Carga.
S3	Amazon Simple Storage Service.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com